

EstateAI — How It Works: Flow, Architecture, Stack and Debugging

Oct 5, 2026 · @Rupam

1. Overview

EstateAI is an India-first property platform (Pune, Bengaluru, Mumbai; prices in INR). Buyers search in plain English, agents publish listings with AI-written descriptions, and every listing has an assistant that answers only from that listing’s facts and PDFs. It is built as **six Python microservices behind a .NET API gateway**, with a **React single-page app** in front. It runs locally on Docker with Azure emulators, so the same code deploys to Azure.

Three ideas explain most of the code:

- 1. **One door in.** The browser only talks to `/api/*`, which reaches the YARP gateway on port 8080. The gateway checks the token, applies rate limits and caching, retries safe calls, and forwards to the owning service. `/internal/*` routes are never exposed.
- 2. **Each service owns its data.** Identity, listing, search and ai each have their own Postgres database; engagement data and AI logs live in Cosmos DB. A service never reads another service’s database. It calls an `/internal/*` HTTP endpoint, or reacts to an event on Azure Service Bus.
- 3. **AI is grounded, optional and replaceable.** Only the `ai` service talks to Google Gemini. Every AI feature has a timeout, a non-AI fallback, a runtime on/off switch, deterministic guardrails, and a fake mode (`AI_FAKE=true`) so the stack runs offline and tests never call the real API.

What people can do, and which service does it:

Who	Can	Services involved
Anyone	Search by filters, sort, map area, or a sentence ("2 BHK under 80 lakh near a metro")	search, ai (parsing)
Anyone	Open a listing, ask its Q&A assistant, message the agent	listing, ai, engagement, notification

Who	Can	Services involved
Signed-in buyer	Save homes and searches, apply to become an agent, delete the account	engagement, identity
Verified agent	Create, edit, publish, duplicate, archive listings; upload photos and PDFs; generate or improve descriptions; read enquiries	listing, ai, engagement
Admin	Verify agents, suspend people, take listings down, watch AI cost and quality, switch AI features, read success metrics	identity, listing, ai, engagement

How to read this document. Sections 2–4 give the stack, the architecture and the folder map. Sections 5–11 follow each flow end to end with the exact files and functions involved. Sections 12–14 cover the shared patterns, the data model and the frontend. Sections 15–18 cover running, debugging, testing and deploying. Paths are relative to the repository root (`D:\Real Estate`).

2. Tech stack

Every layer is a mainstream, open-source choice with an Azure-managed equivalent; the decision records (ADRs) in `docs/decisions/` explain each choice.

Frontend (`web/`)

Piece	Version	What it does here	Why (ADR)
React	19.1	UI components	Product choice (0009)
TypeScript (strict)	5.9	Types for every API shape (<code>web/src/lib/types.ts</code>)	Catch contract drift at build time
Vite	7.1	Dev server on <code>:5173</code> with <code>/api</code> proxy to the gateway; production build to <code>web/dist</code>	Fast, static output for Static Web Apps (0009)
React Router	7.8	Client-side routes (<code>web/src/App.tsx</code>)	Standard SPA routing
TanStack Query	5.85	Server state: caching, refetch, infinite "Show more"	No hand-written fetch state

Piece	Version	What it does here	Why (ADR)
Tailwind CSS	4.1	Styling with design tokens in <code>web/src/index.css</code>	Consistent look, no CSS files per component
MapLibre GL JS	5.24	Search map, listing pin, editor pin picker (lazy-loaded)	Open source, no key (0018)
OpenFreeMap tiles	—	Free OpenStreetMap vector tiles	No key, no usage limits (0018)
Playwright + axe-core	1.63 / 4.13	Browser matrix and WCAG 2.1 AA checks (<code>web/e2e/</code>)	NFR-6, NFR-11
nginx	1.27	Serves the built SPA in Docker and proxies <code>/api</code> to the gateway (<code>web/nginx.conf</code>)	Same-origin cookies and SSE

Backend services (`services/`*, `libs/common/`)

Piece	Version	What it does here
Python	3.12 (images)	All six domain services (ADR-0008)
FastAPI + Uvicorn	≥ 0.115	HTTP APIs, dependency injection for auth, OpenAPI at <code>/docs</code> locally
Pydantic v2 + pydantic-settings	≥ 2.8	Request/response models, structured AI output, settings from env
SQLAlchemy 2 (async) + asyncpg	≥ 2.0 / 0.29	Postgres access
Alembic	≥ 1.13	Schema migrations per Postgres service
pgvector	≥ 0.3	768-dim vectors for semantic search and Q&A retrieval (ADR-0002)
azure-servicebus	≥ 7.12	Publish/consume integration events (ADR-0011)
azure-cosmos	≥ 4.7	Engagement data, AI request log, product events (ADR-0012)

Piece	Version	What it does here
azure-storage-blob	≥ 12.22	Photos and PDFs in Blob Storage (Azurite locally)
redis-py	≥ 5.0	Cache L2, OTP and magic-link codes, token revocation, idempotency, feature flags
PyJWT[crypto]	≥ 2.9	RS256 tokens, JWKS, Google ID-token verification (ADR-0015, 0017)
google-genai	1.30–2.x	Gemini generation and embeddings, only in <code>services/ai</code> (ADR-0013)
Jinja2 + PyYAML	≥ 3.1 / 6.0	Versioned prompt files with front-matter
pypdf	≥ 5.0	Text extraction from uploaded PDFs
Pillow + pillow-heif	≥ 10.4 / 0.18	Re-encode photos to WebP, strip EXIF/GPS, accept iPhone HEIC
sse-starlette	≥ 2.1	Streaming Q&A answers (ADR-0006)
httpx	≥ 0.27	Service-to-service calls (wrapped by <code>ResilientClient</code>)
structlog	≥ 24.4	JSON logs with request ids
OpenTelemetry	≥ 1.27	Traces to the Aspire Dashboard locally, App Insights in Azure
aiosmtp	≥ 3.0	Sign-in and enquiry emails (Mailpit locally)
pytest, pytest-asyncio, fakeredis, ruff, mypy	—	Tests, Redis fake, lint, types

Gateway (`services/gateway` , .NET 10 / C#, ADR-0014)

Package	Version	What it does here
Yarp.ReverseProxy	2.3	Route table and clusters in <code>src/Gateway/appsettings.json</code>

Package	Version	What it does here
JwtBearer	10.0.12	Verifies RS256 tokens against the identity service's JWKS
RedisRateLimiting.AspNetCore	1.2.1	Sliding-window rate limits per user or IP, shared across replicas
OutputCaching.StackExchangeRedis	10.0.12	Caches anonymous GETs (search, listing detail, media)
Microsoft.Extensions.Http.Resilience	10.10	Retries and circuit breakers (Polly) for upstream calls
OpenTelemetry	1.19	Gateway traces join the services' traces

Data and infrastructure

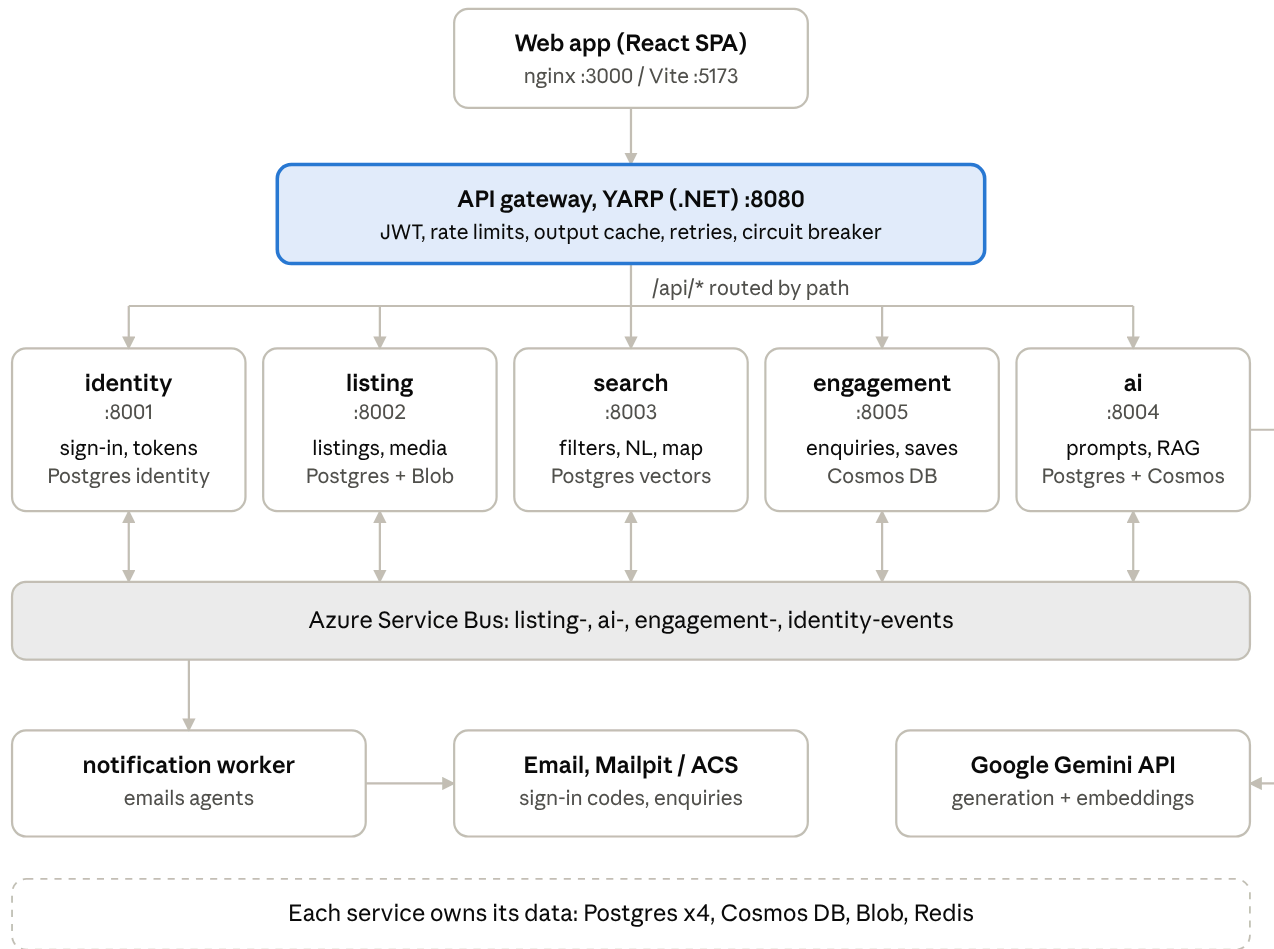
Local (docker compose)	Azure	Used for
<code>pgvector/pgvector:pg16</code>	PostgreSQL Flexible Server 16	identity, listing, search, ai databases
Cosmos DB Linux emulator (vNext)	Cosmos DB for NoSQL, serverless	enquiries, favourites, saved searches, product events, AI log
Service Bus emulator + SQL Server 2022	Service Bus Standard	listing-, ai-, engagement-, identity-events topics
Azurite	Storage account	<code>listing-media</code> container
<code>redis:7-alpine</code>	Azure Cache for Redis	caches, rate limits, revocation, flags
Mailpit	Communication Services Email	outgoing email
Aspire Dashboard	Application Insights + Log Analytics	traces, logs
—	Container Apps, Static Web Apps, Front Door, Key Vault, ACR	hosting and secrets (<code>infra/azure/*.bicep</code>)

AI models (config only, never hard-coded): `gemini-3.1-flash-lite` for query parsing, `gemini-3.8-flash` for descriptions, "improve my text" and Q&A, `gemini-3.1-pro-preview` as the offline eval judge, `gemini-embedding-001` at 768 dimensions.

3. Architecture

Seven backend processes plus the web app: one gateway, five HTTP services and one worker, each with its own data. They talk in two ways only: synchronous `/internal/*` HTTP for questions that need an answer now, and Service Bus events for "this happened" news that others react to later.

Everything enters through the gateway; services share news over Service Bus



architecture · web, gateway, 5 services, worker, Service Bus

The browser reaches only the gateway; the gateway routes `/api/*` by path to one of five services. Services announce changes on Service Bus (double arrows: each publishes and/or consumes), the notification worker turns enquiry events into email, and only the ai

service calls Gemini. Synchronous `/internal/*` calls between services are listed in the table below.

Services

Service	Local port	Owns	Responsibility	Calls (HTTP)	Publishes / consumes
gateway (C#)	8080	rate-limit counters, output cache (Redis)	Only public entry: routes, JWT check, revocation check, rate limits, output cache, retries, circuit breakers, error envelope, blocks <code>/internal/*</code>	all HTTP services	—
identity	8001	Postgres <code>identity</code> : users, signing keys, refresh tokens, admin_actions, outbox; Redis: OTP and magic-link codes	Sign-in (email code, magic link, Google), JWT issue + JWKS, roles, agent applications, suspension, account deletion and purge	—	publishes <code>identity-events</code>
listing	8002	Postgres <code>listing</code> : listings, images, documents, moderation_actions, outbox; Blob <code>listing-media</code>	Listing aggregate, validation, publish rules, photos, PDFs, moderation, fact sheet for AI, dev seed	identity (seed), ai (fair-housing check)	publishes <code>listing-events</code> ; consumes <code>ai-events</code> , <code>identity-events</code>
search	8003	Postgres <code>search</code> : <code>search_listings</code> read model + vectors; POI list (CSV)	Projection of published listings, classic search, map area, <code>near</code> ranking, NL hybrid search	ai (parse, embed)	consumes <code>listing-events</code>

Service	Local port	Owns	Responsibility	Calls (HTTP)	Publishes / consumes
ai	8004	Postgres <code>ai</code> : document_chunks + vectors, outbox; Cosmos <code>ai-requests</code>	Only Gemini caller: prompts, guardrails, describe, improve, Q&A (RAG + SSE), PDF ingestion, AI logging and cost, admin AI dashboard, feature flags	listing (facts)	consumes <code>listing-</code> <code>events</code> , <code>identity-</code> <code>events</code> ; publishes <code>ai-</code> <code>events</code>
engagement	8005	Cosmos: enquiries, favourites, saved- searches, product- events	Enquiries (with consent), favourites, saved searches, erasure, retention, success metrics	listing (summary)	publishes <code>engagement-</code> <code>events</code> ; consumes <code>identity-</code> <code>events</code>
notification	— (worker)	nothing	Emails agents about new enquiries	identity, engagement (internal)	consumes <code>engagement-</code> <code>events</code>
web	3000 (nginx) / 5173 (Vite dev)	browser storage only	React SPA	gateway via <code>/api</code>	—

Rules that hold everywhere (from `CLAUDE.md` §7, enforced by review, `scripts/check_api_sync.py` and gateway tests):

- A service never reads another service's database or Cosmos container.
- Events carry ids and non-personal data only; consumers are idempotent because delivery is at-least-once.
- `/internal/*` endpoints are never added to the gateway's route table, so the browser cannot reach them (a gateway test checks this).
- Every public endpoint ships with the UI that calls it and a gateway route with the right auth, rate-limit, cache and timeout policies.
- Shared code lives in `libs/common/estate_common` only if it has no domain logic (auth, events, outbox, cache, HTTP resilience, flags, logging, telemetry).

Who calls whom over HTTP (all `/internal/*` , through `ResilientClient`)

Caller	Endpoint	Why
search	<code>ai /internal/nl-parse</code>	Turn a sentence into structured filters (Gemini Flash-Lite, 2 s budget)
search	<code>ai /internal/embeddings</code>	Vector for a query or a listing
ai	<code>listing /internal/listings/{id}/facts</code>	Citable fact sheet S1...Sn (no address or agent contact)
listing	<code>ai /internal/guardrails/listing-text</code>	Fair-housing check on agent text before publishing (no LLM)
listing (seed)	<code>identity /internal/users/by-email/{email}</code>	Find the dev agent
engagement	<code>listing /internal/listings/{id}/summary</code>	Title, status and agent of a listing being enquired about
notification	<code>identity /internal/users/{id}</code> , <code>engagement /internal/enquiries/{agent}/{id}</code>	Agent's email and the enquiry text to put in the email
gateway	<code>identity /.well-known/openid-configuration</code> and <code>/jwks.json</code>	Public keys to verify tokens

4. Repository layout

One repository holds every service, the shared library, the web app, infrastructure and the specs. Every Python service has the same shape, so once you know one you can find your way in all of them.

Top level

Path	What it is
<code>CLAUDE.md</code>	Rules for anyone (human or AI) changing the code: boundaries, patterns, commands
<code>docker-compose.yml</code>	The full local stack: emulators, databases, all services, web

Path	What it is
<code>.env.example</code>	Every setting with local emulator values; copy to <code>.env</code> (never commit <code>.env</code>)
<code>.github/workflows/ci.yml</code>	CI: lint, tests per service, migrations up/down/up, gateway tests, web build, API contract, Bicep compile, Docker builds, end-to-end + Playwright
<code>docs/</code>	Specs and the project's memory: requirements, design, architecture, tasks, ADRs, runbook, security, perf
<code>libs/common/estate_common/</code>	Shared, domain-free Python code used by every service
<code>services/<name>/</code>	One folder per service (below)
<code>web/</code>	React SPA, Playwright tests, nginx config
<code>infra/local/</code>	<code>postgres/init.sql</code> (creates the 4 databases), <code>servicebus/Config.json</code> (topics and subscriptions for the emulator)
<code>infra/azure/</code>	Bicep templates for Azure (<code>main.bicep</code> + 6 modules, <code>main.prod.bicepparam</code>)
<code>scripts/</code>	<code>smoke_test.py</code> (31 end-to-end checks), <code>check_api_sync.py</code> (UI ↔ gateway ↔ backend contract), <code>gen_api_reference.py</code> (writes <code>docs/api-reference.md</code>)
<code>perf/</code>	<code>seed_search.py</code> (100k synthetic listings), <code>k6/search.js</code> (load test)
<code>evals/</code>	<code>run_evals.py</code> + <code>search/cases.jsonl</code> (AI quality checks; real-model runs cost money)
<code>.claude/commands/</code>	Slash commands: <code>/next-task</code> , <code>/new-adr</code> , <code>/new-prompt-version</code> , <code>/spec-check</code>

Shared library `libs/common/estate_common/`

File	Provides
<code>settings.py</code>	<code>CommonSettings</code> : env vars every service reads (JWT issuer/audience, Redis URL, Service Bus connection, internal service URLs, HTTP resilience defaults)
<code>app.py</code>	<code>create_app()</code> : FastAPI factory with logging, error envelope, telemetry, <code>/health</code> and <code>/health/ready</code>
<code>errors.py</code>	<code>NotFound</code> , <code>Forbidden</code> , <code>ValidationFailed</code> , <code>Conflict</code> , <code>AIUnavailable</code> , <code>DependencyUnavailable</code> ; request-id middleware; the JSON error envelope
<code>auth.py</code>	JWT verification via JWKS, revocation check in Redis, <code>User</code> / <code>Agent</code> / <code>Admin</code> / <code>OptionalUser</code> dependencies
<code>events.py</code>	Event envelope (<code>Event</code>) and the catalogue of topics and event types
<code>messaging.py</code>	<code>EventPublisher</code> (direct publish) and <code>consume_forever</code> (subscription receiver with retry/dead-letter)
<code>outbox.py</code>	<code>add_event</code> + <code>OutboxRelay</code> : transactional outbox for Postgres services
<code>http.py</code>	<code>ResilientClient</code> : timeouts, retries for idempotent calls, circuit breaker
<code>cache.py</code>	<code>TwoLevelCache</code> : in-process L1 + Redis L2, single-flight, invalidation broadcast
<code>idempotency.py</code>	<code>IdempotencyMiddleware</code> for <code>Idempotency-Key</code> on unsafe POSTs
<code>flags.py</code>	<code>FeatureFlags</code> : runtime on/off switches in Redis
<code>db.py</code> , <code>migrations.py</code> , <code>logging.py</code> , <code>telemetry.py</code>	SQLAlchemy engine/session, Alembic helper, structlog JSON setup, OpenTelemetry setup

Inside a Python service (example: `services/listing/`)

Path	What it is
<code>app/main.py</code>	The FastAPI app: lifespan (consumers, relays, background loops), every endpoint
<code>app/config.py</code>	The service's settings class (extends <code>CommonSettings</code>)
<code>app/models.py</code>	SQLAlchemy tables (Postgres services)
<code>app/schemas.py</code>	Pydantic request/response models
other <code>app/*.py</code>	Domain modules: <code>rules.py</code> , <code>images.py</code> , <code>mapping.py</code> , <code>money.py</code> , <code>blob.py</code> , <code>seed.py</code> (listing); <code>signin.py</code> , <code>otp.py</code> , <code>tokens.py</code> (identity); <code>ranking.py</code> , <code>pois.py</code> , <code>ai_client.py</code> (search); <code>privacy.py</code> , <code>product_metrics.py</code> , <code>store.py</code> (engagement)
<code>migrations/versions/</code>	Alembic revisions (<code>0001_initial.py</code> , <code>0002_....py</code>)
<code>tests/</code>	pytest unit tests (no Docker needed)
<code>Dockerfile</code> , <code>requirements.txt</code> , <code>pytest.ini</code> , <code>alembic.ini</code>	Build, dependencies, test path setup, migration config

AI service extras (`services/ai/app/`): `llm/base.py` (the `LLMClient` protocol), `llm/gemini_client.py` (the only Gemini code), `llm/fake.py` (offline client), `embeddings.py` , `prompts/*.v1.md` + `prompts/loader.py` , `features/` (`nl_search.py` , `rules.py` , `describe.py` , `improve.py` , `qa.py` , `schemas.py`), `guardrails.py` , `ingestion.py` , `telemetry_store.py` (Cosmos AI log), `pricing.py` , `dashboard.py` .

Gateway (`services/gateway/`): `src/Gateway/Program.cs` (pipeline), `src/Gateway/appsettings.json` (routes, clusters, rate limits), `Infrastructure/` (`EdgeAuthentication.cs` , `RateLimiting.cs` , `Caching.cs` , `Resilience.cs` , `Middleware.cs` , `ErrorEnvelope.cs`), `tests/Gateway.Tests/GatewayTests.cs` .

Web (`web/src/`): `App.tsx` (routes), `pages/` (Home, Search, ListingDetail, ListingEditor, AgentListings, Saved, Account, Login, LoginMagic, Privacy, Admin), `components/` (search box, chips, filter panel, cards, maps, Q&A panel, description tools, photo manager, admin/), `lib/` (`api.ts`, `sse.ts`, `auth.tsx`, `types.ts`, `features.ts`, `analytics.ts`), `index.css` (tokens). Tests in `web/e2e/`.

5. Life of a request

Every browser call passes the same 12 stages, and the same request id (`X-Request-Id`) is stamped on every log line, span and error along the way. That id is your main debugging handle (section 16).

Example: an anonymous buyer opens `/search?city=Pune`, which calls `GET /api/v1/search?city=Pune`.

1. **Browser → same origin.** `web/src/lib/api.ts` sends `fetch("/api/v1/search?city=Pune")` with `credentials: "same-origin"`. If the user is signed in, it adds `Authorization: Bearer <access token>` from memory (never `localStorage`).
2. **Web server.** In Docker, `nginx` (`web/nginx.conf`) proxies `/api/` to `gateway:8080` with buffering off (so Q&A streaming works). In dev, Vite proxies `/api` to `localhost:8080` (`web/vite.config.ts`). Either way the SPA and API share one origin, so cookies are first-party and there is no CORS.
3. **Request id** (`RequestIdMiddleware`). Reuses an incoming `X-Request-Id` if it is safe, otherwise creates one, and echoes it in the response.
4. **Exception handler + security headers.** Any crash becomes the JSON error envelope; responses get `nosniff`, frame and referrer headers.
5. **CORS** for the listed dev origins (`Cors:Origins`).
6. **Authentication** (`EdgeAuthentication.cs`). If there is a Bearer token, ASP.NET `JwtBearer` verifies the RS256 signature with keys from `identity/.well-known/jwks.json`, plus issuer `estateai-identity`, audience `estateai` and expiry. No token = anonymous.
7. **Edge token check** (`EdgeTokenMiddleware`). A present but invalid token is rejected with `401 token_invalid` even on public routes. Then Redis is checked for `jwt:deny:<jti>` (logged out) and `jwt:revoked_before:<user id>` (signed out everywhere, role changed, suspended).

8. **Authorization.** The route's `AuthorizationPolicy` (`authenticated` , `agent` = agent or admin, `admin`) is enforced. Failure returns `401` or `403` in the envelope before the service is ever called.
 9. **Rate limiting** (`RateLimiting.cs`). A global 600 requests/minute per IP, plus named sliding-window policies stored in Redis per user (or IP when anonymous): `n1_search` 30/min, `qa` 20/min, `describe` 10/min (also used by `improve`), `enquiry` 5/hour, `auth` 10/min, `events` 120/min. Over the limit returns `429 rate_limited` with `Retry-After` .
 10. **Output cache** (`Caching.cs`). Anonymous GETs on cached routes are served from Redis: `public-short` 30 s varied by query string (search, listing detail), `suggestions` 5 min, `media` 6 h. Requests with `Authorization` or responses setting cookies are never cached.
 11. **Timeouts + proxy.** Default 30 s; `ai-stream` 90 s (describe, improve, Q&A); `upload` 120 s. YARP picks a healthy destination (active health checks hit `/health/ready` every 10 s) and forwards through `ResilientForwarderHttpClientFactory` . That retries GETs twice with exponential backoff and jitter (200 ms base) on 502/503/504 or connection errors, and opens a circuit breaker per service after 50% failures over 30 s with at least 10 calls, for 15 s. An open circuit or unreachable service returns `503 service_unavailable` .
 12. **The service.** FastAPI runs its own request-id middleware (same id), validates the request with Pydantic, verifies the token again through `estate_common.auth` (defence in depth), runs the handler, and logs one `request` line with method, path, status and duration. Domain errors become the same envelope: `{"error": {"code", "message", "details": [{"field", "issue"}], "request_id"}}` .
- Unknown paths**, including any `/internal/*` , hit the gateway's fallback and get `404 not_found` ("Unknown route").

Status codes you will see

Code	Envelope code	Meaning
401	<code>unauthorized</code> / <code>token_invalid</code>	No valid session; the SPA refreshes once, then asks to sign in
403	<code>forbidden</code>	Signed in but not allowed (role or ownership)
404	<code>not_found</code>	Unknown route, or a listing that isn't published (to non-owners)

Code	Envelope code	Meaning
409	<code>conflict</code>	State clash (e.g. take down a listing that isn't live), or a concurrent duplicate POST
422	<code>validation_error</code>	Bad input; <code>details</code> names each field (the editor highlights them)
429	<code>rate_limited</code>	Slow down; <code>Retry-After</code> says how long
503	<code>service_unavailable</code> / <code>dependency_unavailable</code> / <code>ai_unavailable</code>	A service, a dependency or the AI is down; <code>Retry-After: 10</code>
504	—	The route's timeout passed

6. Sign-in, tokens and roles

Three sign-in methods all end the same way: the identity service issues a 15-minute RS256 access token (kept in browser memory) and a 30-day rotating refresh token (an httpOnly cookie). Every other service only verifies tokens, using the identity service's public keys (ADR-0009, 0015, 0017).

Method 1 — email code and magic link (`services/identity/app/otp.py` , `signin.py`)

1. `Login.tsx` posts the email to `POST /api/v1/auth/otp/request` (rate limit `auth` , 10/min).
2. `otp.create_code` stores a SHA-256 hash of a random 6-digit code in Redis (`otp:<email>` , 10 min). `signin.create_magic_token` stores a hash of a 256-bit random token (`magic:<sha256>` , 15 min).
3. One email goes out with both: the code, and a link `WEB_BASE_URL/login/magic#token=...` . The token sits in the URL fragment, which browsers never send to servers, so it can't leak into logs. Locally the email lands in Mailpit (:8025) and the response also includes `dev_code` .
 - 4a. **Code:** `POST /auth/otp/verify` compares with `hmac.compare_digest` ; after 5 wrong tries the code is dead.
 - 4b. **Link:** `LoginMagic.tsx` reads the fragment, removes it from the address bar, and posts it to `POST /auth/magic/verify` . Redis `GETDEL` consumes it atomically, so it works once.

Method 2 — Google (only when `GOOGLE_CLIENT_ID` is set). `GET /auth/providers` returns the client id; `GoogleButton.tsx` then loads Google Identity Services and receives an ID token. `POST /auth/google` verifies its RS256 signature against Google's JWKS, the audience, the issuer and `email_verified`. No Google access or refresh token is ever requested.

After any method `_user_for_email` finds or creates the account (new accounts are `buyer`) and rejects suspended ones. `_session_for` then:

- signs an access token with the active key (`tokens.issue_access_token`): claims `sub`, `role`, `agent_verified`, `iss`, `aud`, `iat`, `exp` (15 min), `jti`, header `kid`;
- creates a refresh token: random, stored only as a hash in `refresh_tokens` with a `family_id`, and set as cookie `estate_refresh` (httpOnly, SameSite=Strict, path `/api/v1/auth`, Secure outside local).

Keeping the session (`web/src/lib/api.ts`, `auth.tsx`)

- On page load `refreshSession()` calls `POST /auth/refresh`; the cookie comes along automatically.
- Any API call that returns 401 triggers one refresh (shared by concurrent callers) and one retry with the same idempotency key.
- Refresh **rotates** the token: the old row is marked replaced and a new one issued in the same family. Presenting an already-rotated token means it was stolen, so the whole family is revoked (reuse detection).

Ending sessions and revocation (Redis, checked by the gateway and every service)

Action	Endpoint	What it writes
Sign out	<code>POST /auth/logout</code>	Revokes the refresh family; <code>jwt:deny:<jti></code> until the token would expire
Sign out everywhere	<code>POST /auth/logout-all</code>	Revokes all refresh tokens; <code>jwt:revoked_before:<user> = now + 1</code>
Role change, suspension, account deletion	<code>admin / DELETE /me</code>	Same <code>revoked_before</code> cut-off, so old tokens die at once

The cut-off is one second ahead so tokens issued in that same second die too. A token minted right after a cut-off is stamped `iat = cut-off ( issued_at_floor )`, so a fresh sign-in isn't rejected along with them.

Keys. Each `signing_keys` row is a 2048-bit RSA key pair with a `kid`. The newest non-retired key signs; keys rotate every 30 days or on demand (`POST /api/v1/admin/keys/rotate`); retired keys stay in JWKS until their tokens expire. Verifiers cache JWKS for 5 minutes and refetch when they see an unknown `kid`. Locally the private key lives in Postgres; production should move it to Key Vault (task T4.14).

Roles (`estate_common/auth.py` dependencies + gateway policies)

Role	How you get it	Can
buyer	Default for every new account	Save homes and searches, enquire, apply to be an agent
agent (unverified)	<code>POST /me/agent-request</code> with name, phone, agency, RERA agent number	Open agent pages, but creating listings is refused until verified
agent (verified)	Admin ticks "Verified agent" on Admin → People	Create and manage own listings, use the AI writing tools
admin	Set by another admin (dev seed: <code>admin@example.com</code>)	Everything, plus admin pages

Local test accounts: `agent@example.com` (verified agent) and `admin@example.com`, created at identity start-up when `APP_ENV=local`.

7. Listing lifecycle

The listing service is the source of truth for listings; every other service learns about a listing only through its events or its internal endpoints. All code below is in `services/listing/app/main.py` unless noted.

Statuses

Status	Set by	Visible to buyers	Agent can edit
<code>draft</code>	create, duplicate	no	yes

Status	Set by	Visible to buyers	Agent can edit
published	agent publish, admin restore, reinstated agent	yes	yes
unpublished	agent unpublish	no (404)	yes
archived	agent archive, account deletion	no; also gone from the agent's list	—
removed	admin takedown	no	no (403 with a message)
suspended	the agent's account was suspended	no	no

Create and edit

1. `ListingEditor.tsx` posts the form to `POST /api/v1/listings` with an `Idempotency-Key`, so a double-click creates one listing (`IdempotencyMiddleware`). Only verified agents (or admins) may create.
2. `ListingFields` (`Pydantic`, `schemas.py`) validates every field: listing/property type, price in **rupees** (`price_inr`), BHK, bathrooms, balconies, carpet and built-up area (carpet may not exceed built-up), floor, facing, furnishing, parking (covered and open), pet policy, possession (`ready_to_move`, `under_construction` or a `YYYY-MM` date), property age, amenities (fixed list of 12), address, locality, city, 6-digit pincode, latitude/longitude, RERA id.
3. `_apply` converts rupees to integer **paise** (`price_minor`, `deposit_minor`, `maintenance_minor`; `money.py`). Money is never a float. Responses return `{amount_minor, currency, display}` with lakh/crore formatting (`₹80 L`, `₹1.25 Cr`).
4. `PATCH` merges the change into the current fields and re-validates the whole listing. Errors come back as `details: [{field, issue}]` and the editor highlights each field.

Publish (`publish_listing`)

1. `_ensure_editable`: owner or admin, and not `removed` / `suspended`.
2. `rules.publish_problems` lists every missing field at once: description of at least 50 characters, pincode, possession; carpet area, bathrooms and furnishing (except plots, which need an area instead); deposit for rentals; RERA id when `rules.rera_required`

(a sale that is under construction or has a future possession date); both or neither of latitude and longitude.

3. `_fair_housing_problems` sends the title and description to `ai /internal/guardrails/listing-text`, a word-pattern check with no LLM. A sentence that combines a protected group (religion, caste, community, diet, marital status, bachelors, ...) with exclusion words ("only", "not allowed", "preferred", ...) blocks publishing. If the ai service is down, publishing is allowed and a warning is logged.
4. Any problem returns `422` with one detail per field. Otherwise status becomes `published`, `published_at` is set the first time, and `_stage_state` adds a `listing.published` event to the **outbox in the same transaction** (section 12). After commit the listing cache is invalidated.

Photos (FR-1.2): `POST /listings/{id}/images` (multipart, max 15 MB, 30 per listing) → `images.process_image` opens JPEG/PNG/WebP/HEIC with Pillow (+ pillow-heif), applies EXIF rotation, drops all metadata including GPS, and writes two WebP files to Blob: 1600 px display and 400 px thumbnail (`listings/<id>/images/<image id>.webp` and `_thumb.webp`). `PUT .../images/order` reorders (the first photo is the cover and search thumbnail); `DELETE .../images/{image_id}` removes one. Changes to a live listing re-emit `listing.updated` so search gets the new thumbnail. Photos are served publicly through `GET /api/v1/media/<key>` (cached 6 h at the gateway).

Documents (FR-1.3): `POST /listings/{id}/documents` accepts only PDFs (checks the `%PDF-` magic bytes, max 20 MB, 10 per listing). It stores the file in Blob, creates a `listing_documents` row with status `processing`, and emits `listing.document_uploaded`. The ai service reads, chunks and embeds it (section 9) and replies with `ai.document_processed`, which sets the status to `ready` or `failed`. The editor polls every 4 s while anything is processing. Owners can download (`GET .../documents/{doc}/file`) or delete (`DELETE`, which emits `listing.document_deleted` so ai drops the chunks). Buyers never get the files, only cited answers.

Other actions

- `POST .../unpublish` hides a live listing; `POST .../archive` hides it for good; both emit `listing.unpublished` if it was live.
- `POST .../duplicate` copies every field into a new draft titled "Copy of ..." (photos and PDFs are not copied).
- `GET /api/v1/listings/{id}`: owners and admins always see fresh data; everyone else gets the published version from a two-level cache (10 s in memory, 5 min in Redis) or 404.

Moderation and suspension (FR-7.1): admins call `POST`

`/api/v1/admin/listings/{id}/takedown` or `/restore` with a reason of at least 5 characters. Each action is written to `moderation_actions` (admin, action, reason, time) and emits `listing.unpublished` or `listing.published`. When identity emits `identity.user_suspended`, `on_identity_event` moves that agent's live listings to `suspended`; `identity.user_reinstated` moves them back; `identity.user_deleted` archives all of them.

For the AI: `GET /internal/listings/{id}/facts` builds the citable fact sheet (`mapping.to_facts`): S1, S2, ... with label and value for every filled field, deliberately excluding the street address and agent contact. `to_snapshot` builds the event payload for search: the searchable fields, coordinates and thumbnail key, with no personal data.

Dev helpers (local only): `POST /internal/dev/seed?count=60` creates synthetic published listings for the dev agent across 16 localities; `POST /internal/dev/republish` re-emits `listing.updated` for every live listing so search can rebuild.

8. Search

Search never reads the listing database: it keeps its own copy of every published listing (a "read model") built from listing events, and answers every search from that copy. Code: `services/search/app/` (`main.py`, `ranking.py`, `pois.py`, `models.py`, `ai_client.py`).

Building the read model (`on_listing_event`)

1. The search service subscribes to `listing-events` (subscription `search`) for `listing.published`, `listing.updated` and `listing.unpublished`.
2. Unpublished, or a status other than `published` → delete the row.
3. Otherwise it asks `ai /internal/embeddings` for a 768-number vector of `embedding_text` ("apartment 2BHK for sale in Kharadi, Pune. <title>. <description> Amenities: ..."). If that fails the row is stored without a vector and only full-text and filters find it.
4. Upsert into `search_listings` by id (`ON CONFLICT DO UPDATE`), which makes redelivered events harmless. Postgres keeps a generated `search_text` tsvector for full-text search.
5. `search_cache.bump_version()` invalidates every cached search result at once.

To rebuild from scratch, call listing `POST /internal/dev/republish`.

Classic search `GET /api/v1/search` (FR-2)

Parameter	Effect
<code>city</code> , <code>listing_type</code> , <code>property_type</code> , <code>furnishing</code> , <code>pet_policy</code>	exact match
<code>locality</code>	case-insensitive contains
<code>bedrooms_min</code> , <code>bedrooms_max</code>	BHK range
<code>price_min</code> , <code>price_max</code>	rupees, converted to paise
<code>amenities</code> (repeatable)	listing must have all of them (Postgres array <code>@></code>)
<code>bbox=minLng,minLat,maxLng,maxLat</code>	only homes on that part of the map (FR-2.3); an invalid box is ignored
<code>near</code> (repeatable: metro, it_park, school, hospital, mall, railway_station, airport)	ranks closer homes higher; never filters
<code>sort</code>	<code>newest</code> (default), <code>price_asc</code> , <code>price_desc</code> , <code>relevance</code>
<code>cursor</code> , <code>limit</code>	<code>limit</code> 1-50 (UI uses 20); <code>cursor</code> = base64 of the offset, returned as <code>next_cursor</code>

With `newest` or a price sort it runs one SQL query, ordered by that key and then by id so pages are stable. With `relevance` or any `near` it runs the scoring path below without a text query. Only the columns a result card needs are loaded (`ranking.RESULT_COLUMNS`): loading the 768-number vector and the description made search CPU-bound (see section 17, load tests). Results are cached in a two-level cache (10 s in memory, 60 s in Redis) keyed on the parsed filters, plus 30 s at the gateway for anonymous users.

Natural-language search POST `/api/v1/search/nl` (FR-3), body `{query, cursor, limit, sort, bbox}`

1. If the `nl_search` flag is on, two calls run **in parallel**: `ai /internal/nl-parse` (Gemini Flash-Lite; Redis cache `nl:<sha256 of the normalised query>` for 24 h; 2 s budget) and `ai /internal/embeddings` for the query (1.5 s budget).
2. The parser returns `SearchFilters`: `is_property_query`, city, locality, listing type, property type, BHK range, price range (rupees), furnishing, pets, amenities, `near` places, soft preferences ("quiet", "lots of light") and assumptions ("Assumed you want

to buy").

- 3. Not a property query ("what's the weather") → an empty result with `is_property_query: false`; the page shows a friendly hint.
- 4. `ranking.hybrid_search`: hard filters in SQL (budget gets 5% tolerance), then up to 200 candidates ordered by vector similarity. With no vector it falls back to Postgres full-text (`websearch_to_tsquery`), and with neither, to newest.
- 5. Each candidate gets a score: $0.55 \times \text{semantic similarity} + 0.25 \times \text{filter fit} + 0.10 \times \text{near} + 0.10 \times \text{freshness}$ (weights in `config.py` / `.env`). Filter fit loses points for going over budget. `near` is 1.0 within 1.5 km of a matching POI, falling to 0 at 5 km (`pois.py` , a 50-row OpenStreetMap-derived list). Freshness = $e^{-(\text{days since publishing} / 30)}$, so a 30-day-old listing scores 0.37.
- 6. Each result carries `match.reasons` such as "Within budget", "In Kharadi", "Has gym", "0.8 km to Kalyani Nagar".
- 7. `meta.mode` is `ai` (parsed), or `fallback` when parsing failed or timed out; then the UI shows "Showing broad matches".

The page (`web/src/pages/Search.tsx`): all state lives in the URL (`q` , filters, `sort` , `view=map` , `bbox`), so any search can be shared or bookmarked. AI results show as an editable sentence (`FilterChips.tsx`); removing a chip, or "Edit filters", converts the interpretation to classic parameters and re-runs **without** an LLM call. The map view (`MapView.tsx`) shows price pins; moving the map shows "Search this area", which sets `bbox` .

9. The AI layer

The ai service (`services/ai/app/`) is the only code that talks to Gemini, and every feature in it follows the same recipe: clean the input, render a versioned prompt, call the model through one interface, check the output with deterministic rules, log the call with tokens and cost, and fall back gracefully on failure.

The building blocks

Piece	File	What it does
LLM interface	<code>llm/base.py</code>	<code>LLMClient</code> protocol with <code>parse()</code> (structured JSON) and <code>stream()</code> (text tokens); <code>CallContext</code> (feature, user, listing); <code>AIRefused</code> , <code>AIOutputInvalid</code>

Piece	File	What it does
Gemini client	<code>llm/gemini_client.py</code>	<code>generate_content</code> with <code>response_json_schema</code> built from a Pydantic model, then Pydantic validation; <code>generate_content_stream</code> for Q&A; <code>thinking_level</code> from prompt front-matter; per-feature timeouts (search 1.5 s, describe/improve/Q&A 30 s); 2 retries on 408/429/5xx; safety blocks become <code>AIRRefused</code>
Fake client	<code>llm/fake.py</code>	Used when <code>AI_FAKE=true</code> (default locally and in all tests): rule-based query parser, a template description, echo rewrite, and a word-overlap Q&A answer with real citations
Embeddings	<code>embeddings.py</code>	<code>GeminiEmbeddings</code> (<code>gemini-embedding-001</code> , 768 dims, L2-normalised) or <code>HashingEmbeddings</code> (offline, deterministic) via <code>EMBEDDING_PROVIDER</code>
Prompts	<code>prompts/<id>.v<N>.md</code> + <code>loader.py</code>	YAML front-matter (id, version, model config key, max tokens, thinking level), <code># System</code> and <code># User</code> sections, Jinja2 with <code>StrictUndefined</code> ; untrusted text goes through the <code>untrusted</code> filter, which escapes tags so it cannot break out of its XML delimiters. Version chosen by <code>AI_PROMPT_VERSION_<ID></code>
Guardrails	<code>guardrails.py</code>	<code>sanitize</code> (normalise, strip control characters, cap length), <code>redact_pii</code> (emails, Indian phone numbers, Aadhaar, PAN → <code>[REDACTED_...]</code>), <code>screen_intent</code> (legal/tax/investment questions), <code>fair_housing_violations</code> , <code>unsupported_numbers</code> , citation checks, "I don't have that information" detection

Piece	File	What it does
AI request log	<code>telemetry_store.py</code> + <code>pricing.py</code>	One Cosmos document per call in <code>ai-requests</code> (partition <code>/feature</code> , 30-day TTL): feature, model, prompt version, status, latency, input/output/thinking/cached tokens, cost in micro-dollars, redacted request and response, feedback
Flags	<code>estate_common/flags.py</code>	<code>ai_describe</code> , <code>ai_improve</code> , <code>listing_qa</code> (and <code>nl_search</code> read by search)

Feature: NL query parsing (`features/nl_search.py`, prompt `nl_search.v1`): sanitise → redact PII → Gemini Flash-Lite returns `SearchFilters` → `post_process` normalises city names, infers the city from a locality, swaps reversed ranges, drops negative numbers, and removes soft preferences that mention protected groups (with an assumption saying so).

Feature: description generator `POST /api/v1/ai/describe` (FR-4; agent only, flag `ai_describe`, rate limit 10/min)

1. Fetch the fact sheet from listing (cached 30 s in memory, 5 min in Redis, dropped on any listing event); check the caller owns the listing.
2. Render `describe.v1` with the facts, image captions, the agent's notes (redacted, marked untrusted), tone (professional, warm, luxury) and target length (60, 120 or 200 words).
3. Gemini returns `{title, description, highlights}`. `describe.check` flags any number not in the facts or notes, discriminatory wording, a title over 80 characters, or not 3–6 highlights.
4. Problems → one regeneration with the problems fed back. Still problems → return the draft with `warnings`, which the editor shows above it.
5. Nothing is saved: the agent edits the draft and clicks "Use this draft", then saves. The listing records `description_ai = true` (shown as "AI-assisted").

Feature: improve my text `POST /api/v1/ai/improve` (FR-4.4; flag `ai_improve`): same pattern with prompt `improve.v1`. It rewrites the agent's own draft in a chosen tone, keeps every fact and adds none; numbers may only come from the draft or the facts.

Feature: listing Q&A `POST /api/v1/listings/{id}/qa` (FR-5; public, flag `listing_qa`, rate limit 20/min, 90 s timeout) streams Server-Sent Events:

1. Fetch facts; a non-published listing is 404 unless you own it.
2. Sanitise and redact the question. `screen_intent` catches advice questions ("is this a good investment?") and answers with a fixed polite refusal, with no LLM call.
3. Retrieve up to 6 document chunks **for this listing only** (`WHERE listing_id = :id`, cosine similarity ≥ 0.3), labelled D1, D2... with file name and pages. If embeddings fail, answer from facts only.
4. Render `qa.v1` (facts S1...Sn, documents D1...Dn, the question; up to 6 earlier turns of history, redacted) and stream Gemini's answer as `event: token` messages.
5. After the stream: `event: citations` (each cited source with its label and an excerpt: the field value, or up to 400 characters of the passage) and `event: done` with `ai_request_id` and a status: `answered`, `unknown` ("I don't have that information" → "Ask the agent instead"), `unverified` (no valid citation), or `refused`. Errors send `event: error`.
6. If the browser disconnects, the loop stops so no more tokens are paid for.

`GET .../qa/suggestions` returns up to 3 starter questions for fields the listing has, plus one about society rules when a document exists. `POST /api/v1/ai/feedback` records 👍/👎 and an optional comment on that request's log entry.

Document ingestion (`ingestion.py`, on `listing.document_uploaded`): download the PDF from Blob → `pypdf` extracts text page by page (max 100 pages) → chunks of about 450 words with 60 words of overlap, remembering page ranges → embed all chunks → delete any old chunks for that document and insert the new ones → add `ai.document_processed` (`ready`) to the outbox in the same transaction. No text (a scanned PDF) or too many pages → `failed` with a reason the agent sees. On `listing.document_deleted` the chunks are deleted.

Cost and quality are visible on Admin → AI (section 11). Evals live in `evals/` (`python evals/run_evals.py --suite search`); with `AI_FAKE=false` they call Gemini and cost money, so state the expected cost before running them.

10. Engagement and notification

The engagement service (`services/engagement/app/`) holds everything a buyer does about listings, in Cosmos DB; the notification worker

(`services/notification/app/main.py`) turns new enquiries into emails.

Enquiry to email (FR-6.3)

1. The buyer fills the form on the listing page (`EnquiryForm` in `ListingDetail.tsx`): name, email, optional phone, message, and a **required consent tick** (DPDP Act). The message is pre-filled with the question when it comes from "Ask the agent instead" (`source: qa_unknown`). An `Idempotency-Key` makes double submits safe; the gateway allows 5 per hour.
2. `POST /api/v1/listings/{id}/enquiries` asks listing `/internal/listings/{id}/summary` ; a listing that isn't published is refused.
3. The enquiry is stored in Cosmos `enquiries` (partition `/agentId`) with `consentAt` , `createdAt` , the buyer id if signed in, and `eventPublished: false` .
4. It publishes `engagement.enquiry_created` carrying **ids only** (enquiry, agent, listing), with event id `enquiry-<id>` so Service Bus duplicate detection drops repeats, then sets `eventPublished: true` . If publishing fails, a relay retries every 30 s any enquiry still unpublished (Cosmos has no cross-container transactions, so the flag on the document acts as its outbox).
5. The notification worker receives the event, fetches the agent's email from identity and the enquiry from `engagement /internal/enquiries/{agent}/{id}` , and sends an email with `Reply-To` set to the buyer (Mailpit locally, Azure Communication Services in production). Any failure abandons the message, so Service Bus retries it and dead-letters it after 5 attempts. Logs never contain the buyer's details.
6. The agent also sees the enquiry in "Messages from buyers" on `/agent/listings` (`GET /api/v1/agent/enquiries`).

Favourites and saved searches (FR-6.2, FR-3.5): `PUT/DELETE`

`/api/v1/me/favourites/{listing_id}` (document id `<user>:<listing>` , partition `/userId`) and `GET/POST/DELETE /api/v1/me/saved-searches` (stores the name, the NL query if any, and the URL filters). A saved search re-runs the way it was made: as a sentence or as plain filters (`Saved.tsx`).

Erasure (FR-6.4, `privacy.py`): on `identity.user_deleted` the service deletes the user's favourites and saved searches; scrubs enquiries they **sent** (name → "Deleted account", email and phone removed, message replaced, buyer id cleared) so the agent still sees that

an enquiry happened; and deletes enquiries they **received** as an agent, because those hold other people’s details. Running it twice changes nothing.

Retention (NFR-10): a daily loop deletes enquiries older than 730 days
(`ENQUIRY_RETENTION_DAYS`).

Success-metric events (requirements §9, `product_metrics.py`): the web app sends anonymous events to `POST /api/v1/events` (rate limit 120/min): `search_performed {mode}` , `listing_viewed {from_search, mode}` , `listing_published {minutes_to_publish}` , `description_saved {edit_ratio}` , `qa_session` , `enquiry_sent {after_qa}` . Each carries only a random per-tab id; the model forbids extra fields, so an email in an event is rejected with 422. They are stored in `product-events` (partition `/day` , 90-day TTL) and rolled up by `GET /api/v1/admin/product-metrics` .

Cosmos containers

Container	Partition key	Why that key	TTL
<code>enquiries</code>	<code>/agentId</code>	The hot query is "all enquiries for this agent"	none (2-year purge loop)
<code>favourites</code>	<code>/userId</code>	Always read per user	none
<code>saved-searches</code>	<code>/userId</code>	Always read per user	none
<code>product-events</code>	<code>/day</code>	Metrics read whole days	90 days
<code>ai-requests</code> (owned by ai)	<code>/feature</code>	Dashboards group by feature	30 days

11. Admin

The `/admin` page (`web/src/pages/Admin.tsx`) has four tabs; each talks to the service that owns the data, and every action that changes someone else's account or listing needs a written reason that is stored in an audit table.

Tab	What admins do	Endpoints	Where it is stored
People	Find by email; filter "awaiting verification" or "suspended"; change role and tick "Verified agent"; suspend or reinstate; see history; rotate sign-in keys	identity: GET /admin/users, POST /admin/users/{id}/role, /suspend, /reinstate, GET /admin/users/{id}/actions, POST /admin/keys/rotate	users, admin_actions (Postgres identity)
Listings	Search by title, locality or id; filter by status; take down or restore with a reason; see history	listing: GET /admin/listings, POST /admin/listings/{id}/takedown, /restore, GET /admin/listings/{id}/moderation	listings, moderation_actions (Postgres listing)

Tab	What admins do	Endpoints	Where it is stored
AI	Per feature per day: requests, error rate, input/output tokens, cost, p50/p95 latency, 👍/👎 (charts + a table view); switch AI features on/off; read answers marked not helpful; export them as eval candidates (JSONL)	ai: GET /admin/ai/metrics?days= , GET/PUT /admin/ai/flags[/name] , GET /admin/ai/feedback , GET /admin/ai/feedback/export	Cosmos ai-requests ; Redis hash feature_flags
Success metrics	Each requirements §9 metric against its target with on/below-target status	engagement: GET /admin/product-metrics?days= ; ai: GET /admin/ai/metrics?days=30	Cosmos product-events , ai-requests

What a suspension does, end to end: identity sets `suspended_at` , revokes every refresh token, writes `jwt:revoked_before` (all access tokens die), records the reason in `admin_actions` , and adds `identity.user_suspended` to its outbox. The listing service receives it and moves the agent's live listings to `suspended` , which emits `listing.unpublished` for each, so search removes them. Sign-in is refused until an admin reinstates, which reverses all of it.

Feature switches (ADR-0019): defaults come from `FEATURE_*` settings; an admin toggle writes the Redis hash `feature_flags` ; every replica re-reads it at most 5 s later. If Redis is down, services fall back to the configured default and never switch a feature *on*. The web

app reads `GET /api/v1/ai/features` and hides switched-off panels (Q&A, description tools) instead of showing errors. With `nl_search` off, the search box still works as a keyword search.

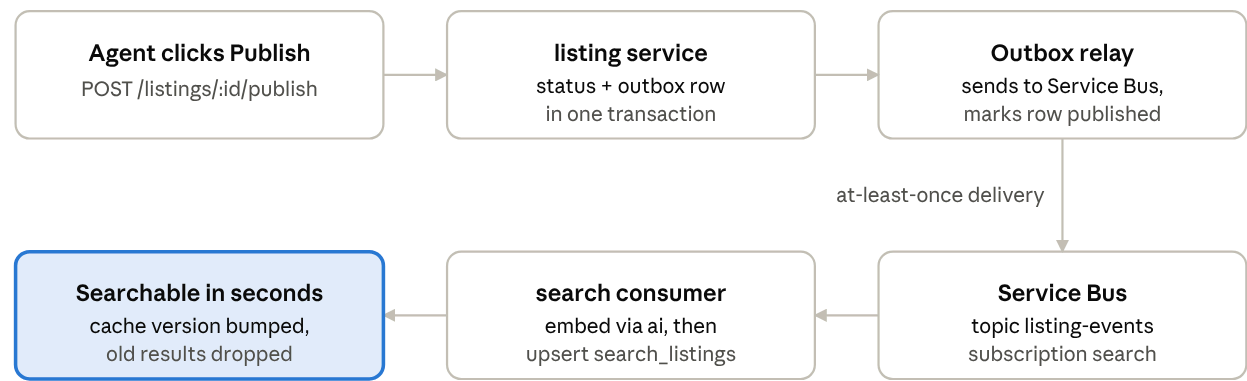
Success metrics and their sources

Metric (target)	Computed from
Search → listing view, NL vs classic (+20%)	<code>listing_viewed</code> with <code>from_search</code> per <code>search_performed</code> , by mode
Share of searches using the NL box ($\geq 40\%$)	<code>search_performed</code> with mode <code>ai</code> or <code>fallback</code>
Median agent time to publish (≤ 10 min)	<code>listing_published.minutes_to_publish</code> (first publish, from draft creation)
AI descriptions kept with $\leq 20\%$ edits ($\geq 60\%$)	<code>description_saved.edit_ratio</code> (word-level edit distance between the accepted AI text and what was saved)
Q&A 👍 ratio ($\geq 80\%$)	AI log feedback, last 30 days
Q&A without contacting the agent ($\geq 50\%$)	<code>qa_session</code> vs <code>enquiry_sent</code> with <code>after_qa</code>
Reported AI problems (< 1 per 1,000)	👎 per Q&A answer (an upper bound on factual errors)

12. Events and production patterns

Services share news through Azure Service Bus topics, and six patterns in `libs/common/estate_common` keep that safe when processes crash, messages repeat or a dependency is down (ADR-0011, 0016).

A publish reaches search through the outbox, never by a direct call



publish → search · 6 steps through the outbox

If the process dies after the commit, the outbox row is still there and the relay sends it on restart; if the event arrives twice, the upsert makes the second one harmless. The same subscription pattern carries every event in the table below.

The envelope (events.py): {id, type, source, subject, time, data, correlation_id} . subject is the aggregate id (usually the listing id); correlation_id is the request id that caused the event, so a consumer's logs join the original trace.

Event catalogue (topology in infra/local/servicebus/Config.json , mirrored in infra/azure/modules/data.bicep)

Topic	Event	Producer	Subscribers	data	What the consumer does
listing-events	listing.published	listing	search, ai	listing snapshot (no PII)	search: upsert + embed; ai: drop cached facts
listing-events	listing.updated	listing	search, ai	listing snapshot	same
listing-events	listing.unpublished	listing	search, ai	listing snapshot	search: delete row; ai: drop cached facts

Topic	Event	Producer	Subscribers	data	What the consumer does
listing-events	listing.document_uploaded	listing	ai	document id, listing id, blob key, file name, kind	extract → chunk → embed → store
listing-events	listing.document_deleted	listing	ai	document id, listing id	delete its chunks
ai-events	ai.document_processed	ai	listing	document id, status, error	set document status
engagement-events	engagement.enquiry_created	engagement	notification	enquiry, agent, listing ids	email the agent
identity-events	identity.user_suspended	identity	listing	user id	hide the agent's live listings
identity-events	identity.user_reinstated	identity	listing	user id	show them again
identity-events	identity.user_deleted	identity	listing, engagement, ai	user id	archive listings; erase engagement data; unlink AI logs

Local emulator limits: message TTL 1 hour and duplicate window 5 minutes. Azure uses 14 days and 10 minutes.

Pattern 1 — Transactional outbox (`outbox.py` ; listing, ai, identity). `add_event(session, ...)` inserts the event into an `outbox` table **in the same transaction** as the state change, so either both happen or neither. `OutboxRelay.run_forever` (a background task in each service) picks pending rows with `FOR UPDATE SKIP LOCKED` (safe with several replicas), sends them in batches with the event id as the Service Bus message id, and marks them published. If Service Bus is down it records the error, backs off up to 30 s and retries. Engagement uses a flag on each Cosmos document instead (section 10).

Pattern 2 — Idempotent consumers (`messaging.consume_forever`). Delivery is at-least-once. Each handler is written so a repeat changes nothing: upsert by id (search), delete-then-insert chunks (ai), state moves only from the expected state (listing), scrub-if-

present (engagement). A handler that raises makes the receiver abandon the message; Service Bus redelivers it and dead-letters it after 3–5 tries (per subscription).

Pattern 3 — Resilient HTTP (`http.ResilientClient`). Timeout (default 5 s); retries with jittered exponential backoff only for idempotent calls (GET/HEAD, or POSTs marked `idempotent=True` such as embeddings); a circuit breaker per dependency that opens after 5 consecutive failures for 30 s. Failures become `DependencyUnavailable` (503 with `Retry-After`).

Pattern 4 — Idempotency keys (`idempotency.IdempotencyMiddleware`). For unsafe POSTs a user could double-submit (create listing, enquiries, saved searches) the SPA sends `Idempotency-Key` . The first request claims the key in Redis (`idem:<service>:<caller hash>:<key>` , 24 h). A repeat with the same body replays the stored response (`Idempotent-Replayed: true`); a different body is 422; a repeat while the first is still running is 409; a 5xx releases the key.

Pattern 5 — Two-level cache (`cache.TwoLevelCache`). L1 in process (bounded, short TTL) + L2 in Redis (shared). Loads are single-flight per key, so a cold key isn't fetched 50 times at once. `invalidate(key)` deletes L2 and broadcasts on Redis pub/sub so every replica drops its L1. Versioned namespaces (`bump_version`) make every cached search result unreachable at once. Redis errors mean "no cache", never a failed request. Used for: listing detail (10 s / 5 min), classic search (10 s / 60 s), AI fact sheets (30 s / 5 min).

Pattern 6 — Health and readiness (`app.create_app`). `/health` = the process is up; `/health/ready` = each declared dependency (Postgres, Redis, Cosmos) answers within 2 s, else 503. The gateway probes `/health/ready` every 10 s and stops routing to unhealthy instances; `GET /health/deep` on the gateway shows every cluster's state.

13. Data model

Four Postgres databases on one server (created by `infra/local/postgres/init.sql`), each changed only through its own service's Alembic migrations, plus Cosmos containers, Blob and Redis keys. Models live in each service's `app/models.py` .

Postgres `identity` (migrations 0001, 0002)

Table	Key columns	Notes
<code>users</code>	id (uuid), email (unique), name, phone, role (buyer/agent/admin), agent_verified, agency_name, rera_agent_id, agent_requested_at, suspended_at, deleted_at, created_at	Phone and name are personal data: never sent to AI or events. On deletion, email becomes <code>deleted-<id>@deleted.invalid</code> and personal fields are cleared; the row is purged after 30 days
<code>signing_keys</code>	kid, private_pem, public_jwk, created_at, retired_at	Private key here only locally (Key Vault in production)
<code>refresh_tokens</code>	id, user_id, family_id, token_hash (unique), expires_at, revoked_at, replaced_by, user_agent	Only hashes stored
<code>admin_actions</code>	id, user_id, admin_id, action (suspend/reinstate/role), detail, reason, created_at	Audit log
<code>outbox</code>	id (= event id), topic, event_type, payload (JSONB), created_at, published_at, attempts, last_error	Same table shape in listing and ai

Postgres `listing` (0001, 0002)

Table	Key columns	Notes
<code>listings</code>	id, agent_id, status, listing_type, property_type, title, description, description_ai, price_minor, deposit_minor, maintenance_minor, currency, bedrooms, bathrooms, balconies, carpet/built-up area, floor, total_floors, facing, furnishing, parking_covered/open, pet_policy, possession, property_age_years, amenities (text[]), address_line, locality, city, pincode, lat, lng, rera_id, published_at, created_at, updated_at	Money in paise (bigint). Indexes on agent_id, status, locality, city

Table	Key columns	Notes
<code>listing_images</code>	<code>id</code> , <code>listing_id</code> , <code>storage_key</code> , <code>thumb_key</code> , <code>caption</code> , <code>position</code>	Ordered by position; first = cover
<code>listing_documents</code>	<code>id</code> , <code>listing_id</code> , <code>storage_key</code> , <code>filename</code> , <code>kind</code> (floor_plan/society_rules/brochure/other), <code>status</code> (uploaded/processing/ready/failed), <code>error</code>	
<code>moderation_actions</code>	<code>id</code> , <code>listing_id</code> , <code>admin_id</code> , <code>action</code> (takedown/restore), <code>reason</code> , <code>created_at</code>	Audit log
<code>outbox</code>	as above	

Postgres `search` (0001, 0002) — `search_listings`: `id` plus the searchable copy (type, title, description, price_minor, BHK, bathrooms, carpet area, furnishing, pet policy, amenities, locality, city, lat, lng, thumbnail key, published_at), `embedding` vector(768) and a generated `search_text` tsvector. Indexes: `ix_search_filters` (city, listing_type, bedrooms, price_minor), GIN on `search_text` and `amenities`, HNSW on `embedding` (cosine), `ix_search_geo` (lat, lng). Rebuildable at any time from events.

Postgres `ai` (0001) — `document_chunks`: `id`, `document_id`, `listing_id`, `filename`, `chunk_index`, `page_from`, `page_to`, `content`, `token_estimate`, `embedding` vector(768) (HNSW cosine), `embedding_model`; plus `outbox`. Rebuildable from the PDFs.

Cosmos `estateai` database: `enquiries`, `favourites`, `saved-searches`, `product-events` (engagement) and `ai-requests` (ai) — partition keys and TTLs in section 10.

Blob container `listing-media` (private): `listings/<listing>/images/<image>.webp`, `..._thumb.webp`, `listings/<listing>/documents/<document>.pdf`. Photos are served through the listing service, PDFs only to their owner.

Redis keys

Key	Owner	Purpose
<code>otp:<email></code> , <code>otp_attempts:<email></code> , <code>magic:<sha256></code>	identity	Sign-in codes and links
<code>jwt:deny:<jti></code> , <code>jwt:revoked_before:<user></code>	identity writes, everyone reads	Revocation

Key	Owner	Purpose
<code>rl:<policy>:<user or ip></code>	gateway	Rate-limit windows
<code>gw-oc:*</code>	gateway	Output cache
<code>listing:*</code> , <code>search:*</code> , <code>facts:*</code> + <code>cacheinv:*</code> channels	listing, search, ai	Two-level cache L2 and invalidation
<code>nl:<sha256></code>	search	Parsed NL queries, 24 h
<code>idem:<service>:<caller>:<key></code>	listing, engagement	Idempotency
<code>feature_flags</code> (hash)	ai writes, ai + search read	Runtime switches

Migrations run automatically before each service starts in docker compose (`alembic upgrade head`), and as Container Apps jobs (`migrate-<service>`) in Azure. CI runs upgrade → downgrade → upgrade against a real pgvector Postgres for all four services. To add one: change `app/models.py`, then from `services/<svc>` run `alembic revision --autogenerate -m "<change>"` with `PYTHONPATH=../../libs/common;.` and check the generated file.

14. Frontend

The web app is a client-rendered React SPA: every page is a component in `web/src/pages/`, server data is fetched through one API client and cached by TanStack Query, and search state lives in the URL.

Routes (`web/src/App.tsx`; `RequireUser`, `RequireAgent`, `RequireAdmin` guard the private ones)

Route	Page	Who	What it shows
<code>/</code>	Home	anyone	Big NL search box with rotating examples; how the Q&A works; neighbourhood links
<code>/search</code>	Search	anyone	NL or classic results, editable chips, filter panel, sort, list/map, save search, "Show more"

Route	Page	Who	What it shows
<code>/listings/:id</code>	ListingDetail	anyone	Photo gallery, facts, description, amenities, map pin, Q&A panel, enquiry form with consent
<code>/login</code> , <code>/login/magic</code>	Login, LoginMagic	anyone	Email → link + code; Google when configured; magic-link landing
<code>/privacy</code>	Privacy	anyone	DPDP notice
<code>/me/saved</code>	Saved	signed in	Saved homes and searches
<code>/account</code>	Account	signed in	Account type, apply to be an agent, sign out (everywhere), delete account
<code>/agent/listings</code>	AgentListings	agent	Own listings with status and "AI-assisted" labels; buyer messages
<code>/agent/listings/new</code> , <code>/:id/edit</code>	ListingEditor	agent	Full form with field errors, map pin picker, photos, PDFs, AI description and improve panels, publish/unpublish/duplicate/archive
<code>/admin?tab=people listings ai metrics</code>	Admin	admin	Section 11

Key modules

File	Role
<code>lib/api.ts</code>	The only HTTP client: <code>api</code> , <code>post</code> , <code>patch</code> , <code>put</code> , <code>del</code> , <code>download</code> . Adds the Bearer token from memory and <code>Idempotency-Key</code> when given; on 401 refreshes once and retries; turns the error envelope into <code>ApiError</code> with field <code>details</code>
<code>lib/auth.tsx</code>	<code>AuthProvider</code> : restores the session from the refresh cookie on load; sign-in by code, magic link or Google; sign out; reloads the user after a role change
<code>lib/sse.ts</code>	<code>postSSE</code> : POST with a streaming body reader for Q&A (EventSource can't POST)

File	Role
<code>lib/types.ts</code>	TypeScript mirrors of the backend schemas
<code>lib/features.ts</code>	<code>useAIFeatures()</code> : which AI panels to show (all on while loading or if ai is down)
<code>lib/analytics.ts</code>	<code>track()</code> fire-and-forget product events; <code>editRatio()</code> for AI drafts
<code>components/MapView.tsx</code> , <code>PinMap.tsx</code> , <code>MapBoundary.tsx</code>	Maps, lazy-loaded; if WebGL is missing the map shows a short note and the page keeps working
<code>components/QAPanel.tsx</code>	Suggestions, streaming answer with caret, clickable sources with excerpts, "Ask the agent instead", 👍/👎 + comment
<code>components/FilterChips.tsx</code> , <code>FilterPanel.tsx</code>	The AI's interpretation as an editable sentence; classic filters
<code>components/DescriptionGenerator.tsx</code> , <code>ImproveText.tsx</code> , <code>PhotoManager.tsx</code>	Editor tools

Security choices: the access token lives only in memory (an XSS can't read a stored token); the refresh cookie is httpOnly and scoped to `/api/v1/auth` ; React escapes all text; no `dangerouslySetInnerHTML` . Google's script loads only when Google sign-in is enabled.

Accessibility (NFR-6): every input has a label; streaming answers sit in `aria-live` ; chips, filters, map pins and photo reordering are keyboard-usable buttons; a skip link; status shown with words and icons, never colour alone; the list is always the primary view and the map an extra. Playwright + axe check WCAG 2.1 A/AA on the main pages in five browser setups.

Design tokens (`web/src/index.css`): limewash `wash #f4f5f1` , indigo `ink #1c2757` , slate, rule, haldi `#e9a31b` (only for "what the AI understood" and keyboard focus), leaf green, danger red; Mukta font; reduced-motion respected.

15. Running it locally

One command starts 16 containers: the Azure emulators, Postgres, Redis, Mailpit, the trace dashboard, seven backend processes and the web app. Docker needs about 6–8 GB of RAM (SQL Server and the Cosmos emulator are the heaviest).

First run

1. `cp .env.example .env` (all values are public emulator credentials; `AI_FAKE=true` means no Gemini key is needed).
2. `docker compose up -d --build` from the repository root. Each Postgres service runs `alembic upgrade head` before starting.
3. Wait until `curl localhost:8080/health/deep` shows `"status":"ok"` (the Service Bus emulator can take a minute; it depends on SQL Server).
4. `curl -X POST "localhost:8002/internal/dev/seed?count=60"` creates 60 published sample listings; within seconds they appear in search.
5. Open `http://localhost:3000` and sign in as `agent@example.com` or `admin@example.com` (the code is shown on screen locally, and also in Mailpit).

Ports

Port	What	Use it for
3000	web (nginx)	The app
8080	gateway	<code>/api/*</code> , <code>/health/deep</code>
8001–8005	identity, listing, search, ai, engagement	Calling a service directly; <code>/docs</code> = Swagger UI (local only)
5432	Postgres	<code>psql -h localhost -U estate</code> (password <code>estate</code>), databases <code>identity/listing/search/ai</code>
6379	Redis	<code>redis-cli</code>
8081 / 1234	Cosmos emulator / its Data Explorer	Browse enquiries, AI logs, product events
5672	Service Bus emulator (AMQP)	Used by services
10000–10002	Azurite (Blob/Queue/Table)	Photos and PDFs

Port	What	Use it for
8025 / 1025	Mailpit UI / SMTP	Read every email sent
18888	Aspire Dashboard	Traces, structured logs, metrics

Development loops

- Python services run with `--reload` and bind-mount `app/` and `libs/common`, so saving a file restarts that service in about a second; no rebuild needed. Rebuild only after changing `requirements.txt` or a Dockerfile.
- The gateway (C#) and the web container are built images: `docker compose up -d --build gateway` or `... web` after changing them.
- For fast frontend work run `cd web && npm install && npm run dev` (port 5173, hot reload, proxies `/api` to the gateway).
- A changed `infra/local/servicebus/Config.json` needs `docker compose restart servicebus`.

Settings worth knowing (`.env`)

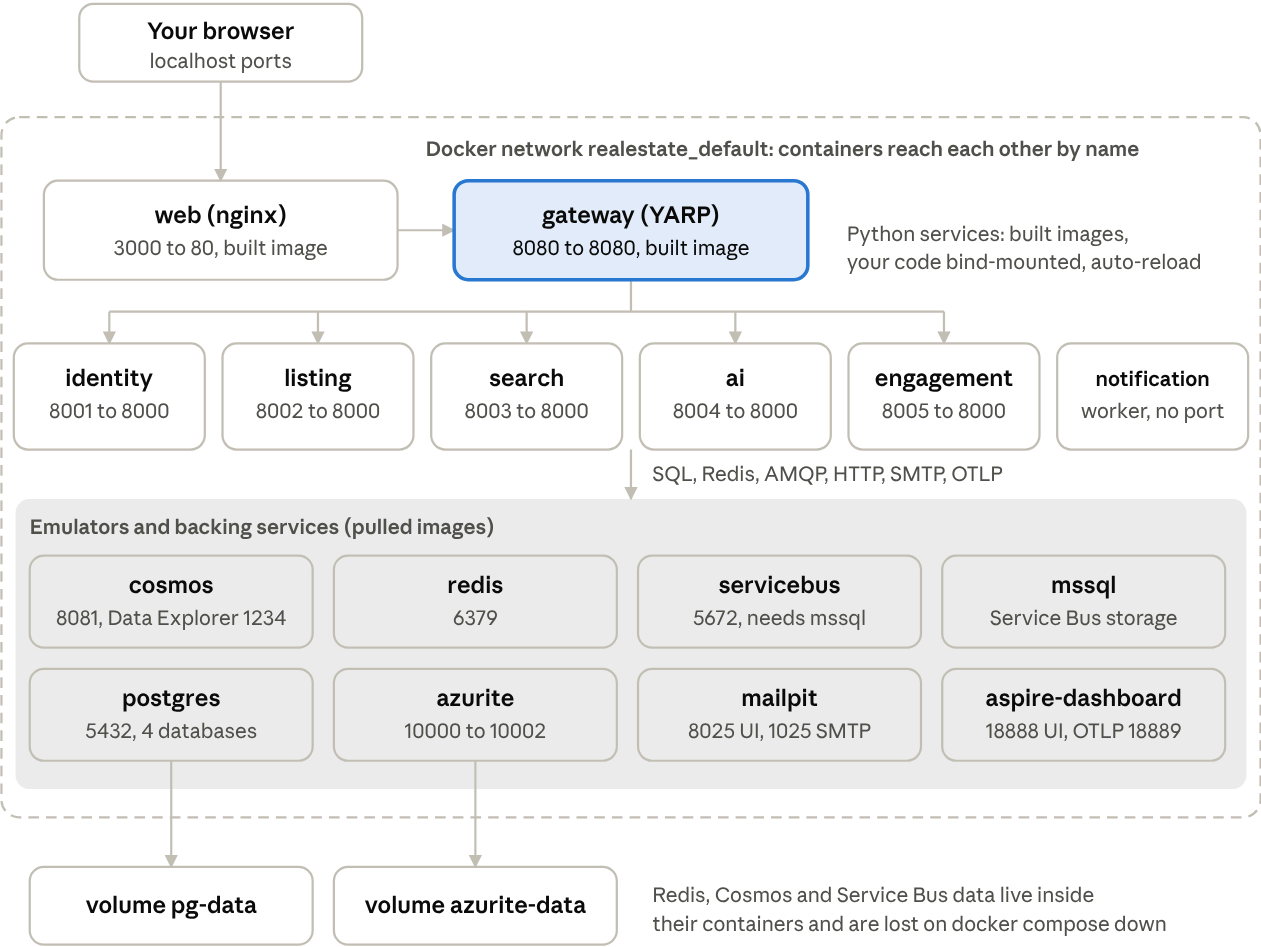
Setting	Default	Effect
<code>APP_ENV</code>	local	<code>local</code> enables dev users, the on-screen sign-in code, <code>/docs</code> and the dev seed endpoints; <code>prod</code> turns them off and makes cookies Secure
<code>AI_FAKE</code> / <code>GEMINI_API_KEY</code>	true / empty	Set <code>AI_FAKE=false</code> and a key to use real Gemini (costs money)
<code>EMBEDDING_PROVIDER</code>	fake	<code>gemini</code> for real embeddings (then re-index with <code>/internal/dev/republish</code>)
<code>AI_MODEL_*</code> , <code>AI_PROMPT_VERSION_*</code> , <code>AI_TIMEOUT_*</code>	see <code>.env.example</code>	Models, prompt versions and timeouts per feature
<code>RANK_W_SEMANTIC/FILTER/NEAR/FRESHNESS</code>	0.55/0.25/0.10/0.10	Search ranking weights
<code>FEATURE_*</code>	true	Default state of each runtime switch
<code>GOOGLE_CLIENT_ID</code>	empty	Shows the Google button when set
<code>ACCOUNT_PURGE_DAYS</code> , <code>ENQUIRY_RETENTION_DAYS</code>	30, 730	Data retention

Reset everything: `docker compose down -v` deletes all volumes (databases, blobs, Redis); then start again from step 2.

15a. Docker architecture

Locally the whole platform is 16 containers defined in one file, `docker-compose.yml`, all on one private Docker network where each container is reachable by its service name. Eight are off-the-shelf images that stand in for Azure services; eight are built from this repository.

16 containers share one Docker network; only two volumes survive a down



docker compose · 16 containers, 1 network, 2 volumes

Your browser enters through `web` on port 3000, which forwards `/api` to the gateway; the gateway routes to the five HTTP services (the notification worker has no port). All of them talk to the emulators by container name. Only Postgres and Azurite write to named volumes.

The containers

Container	Image	Built from	Host port → container port	Stands in for (Azure)	Restart
azurite	mcr.microsoft.com/azure-storage/azurite	pulled	10000-10002 → same	Storage account (Blob, Queue, Table)	none
cosmos	mcr.microsoft.com/cosmosdb/linux/azure-cosmos-emulator:vnext-preview	pulled	8081, 1234 (Data Explorer)	Cosmos DB for NoSQL	none

Container	Image	Built from	Host port → container port	Stands in for (Azure)	Restart
mssql	mcr.microsoft.com/mssql/server:2022-latest	pulled	not published	Nothing — only the Service Bus emulator's storage	none
servicebus	mcr.microsoft.com/azure-messaging/servicebus-emulator	pulled	5672 (AMQP)	Service Bus	none
postgres	pgvector/pgvector:pg16	pulled	5432	PostgreSQL Flexible Server	none
redis	redis:7-alpine (append-only file on)	pulled	6379	Azure Cache for Redis	none
mailpit	axllent/mailpit	pulled	8025 (UI), 1025 (SMTP)	Communication Services Email	none
aspire-dashboard	mcr.microsoft.com/dotnet/aspire-dashboard	pulled	18888 (UI); OTLP 18889 internal only	Application Insights	none
gateway	built	services/gateway/Dockerfile	8080 → 8080	Container App (external ingress)	unless-stopped
identity, listing, search, ai, engagement	built	services/<name>/Dockerfile	8001–8005 → 8000	Container Apps (internal ingress)	unless-stopped
notification	built	services/notification/Dockerfile	none (no HTTP)	Container App worker	unless-stopped
web	built	web/Dockerfile	3000 → 80	Static Web Apps (+ Front Door)	none

All Python services listen on port **8000 inside** their container; only the host-side port differs (8001–8005), so the same image runs unchanged in Azure.

How containers find each other

Compose creates one bridge network (`realestate_default`) and registers each service name in its DNS. That is why every address in `.env` and `CommonSettings` is a service name, not `localhost` :

From	Address used	Setting
any Python service → another	<code>http://identity:8000</code> , <code>http://listing:8000</code> , <code>http://ai:8000</code> , ...	<code>IDENTITY_URL</code> , <code>LISTING_URL</code> , <code>AI_URL</code> , ...
services → Postgres	<code>postgresql+asyncpg://estate:estate@postgres:5432/<db></code>	<code>DATABASE_URL</code> default in each <code>config.py</code>

From	Address used	Setting
services, gateway → Redis	<code>redis://redis:6379/0 / redis:6379</code>	<code>REDIS_URL</code> , <code>Redis__ConnectionString</code>
services → Service Bus	<code>Endpoint=sb://servicebus;...;UseDevelopmentEmulator=true;</code>	<code>SERVICEBUS_CONNECTION</code>
services → Blob	<code>BlobEndpoint=http://azurite:10000/devstoreaccount1</code>	<code>STORAGE_CONNECTION</code>
services → Cosmos	<code>http://cosmos:8081/</code>	<code>COSMOS_ENDPOINT</code>
services → email	<code>mailpit:1025</code>	<code>SMTP_HOST</code> , <code>SMTP_PORT</code>
everything → traces	<code>http://aspire-dashboard:18889 (OTLP)</code>	<code>OTEL_EXPORTER_OTLP_ENDPOINT</code>
gateway → services	<code>http://identity:8000/,...</code>	<code>ReverseProxy:Clusters</code> in <code>appsettings.json</code>
gateway → JWKS	<code>http://identity:8000/.well-known/openid-configuration</code>	<code>Jwt:MetadataAddress</code>
web (nginx) → gateway	<code>http://gateway:8080</code>	<code>web/nginx.conf</code>
Service Bus emulator → its store	<code>mssql</code>	<code>SQL_SERVER</code>

From **your machine** you use `localhost:<host port>` instead (the Ports table in section 15). A process you run on the host can't resolve `listing`; a container reaches your host as `host.docker.internal`.

How the images are built

Python services (six near-identical Dockerfiles):

1. FROM `python:3.12-slim`, with `PYTHONUNBUFFERED=1` (logs appear immediately), `PYTHONDONTWRITEBYTECODE=1` and `PYTHONPATH=/libs/common:/app` (so `import estate_common` and `import app` both work).
2. Copy only `libs/common/requirements.txt` and the service's `requirements.txt`, then `pip install`. Because dependencies are installed before the code is copied, Docker reuses this layer until a requirements file changes, so rebuilds after code edits

take seconds.

3. Copy `libs/common` to `/libs/common`, the service's `app/` to `/app/app`, and (Postgres services) `alembic.ini` + `migrations/`.
4. `EXPOSE 8000`; default `CMD` is plain `uvicorn app.main:app --host 0.0.0.0 --port 8000` (notification: `python -m app.main`).

Their **build context is the repository root** (`build: { context: ., dockerfile: services/<svc>/Dockerfile }`), because every image needs `libs/common` from outside its own folder. The root `.dockerignore` keeps that context small and platform-neutral: no `node_modules`, `__pycache__`, `.pytest_cache`, `.NET bin/` / `obj/`, `web/dist`, `.git` or `.env` (secrets never enter an image).

Gateway (`services/gateway/Dockerfile`, context `services/gateway`): two stages. The `dotnet/sdk:10.0` stage restores packages (cached by the `.csproj` alone) and publishes a Release build; the `dotnet/aspnet:10.0` runtime stage copies only the published output, listens on 8080, enables server GC, and runs as the image's **non-root** user. Its `.dockerignore` excludes `bin/`, `obj/` and the tests.

Web (`web/Dockerfile`, context `web`): two stages. `node:22-alpine` runs `npm install` and `npm run build` (type-check + Vite) to produce `dist/`; `nginx:1.27-alpine` serves `dist/` with `web/nginx.conf`, which falls back to `index.html` for client routes, proxies `/api/` to the gateway with buffering off for streaming, and adds security headers.

`.gitattributes` forces LF line endings so images built on Windows behave like Linux CI builds.

Development mode on top of the images

`docker-compose.yml` reuses one YAML anchor (`x-python-service`) for all Python services and overrides the image defaults for fast local work:

- **Bind mounts:** `./services/<svc>/app` → `/app/app`, `./services/<svc>/migrations` → `/app/migrations`, `./libs/common` → `/libs/common`. The container runs your working copy, not the copy baked into the image.
- **Auto-reload:** `uvicorn ... --reload --reload-dir /app/app --reload-dir /libs/common` restarts the service about a second after you save a file.
- **Migrations first:** identity, listing, search and ai start with `sh -c "alembic upgrade head && uvicorn ..."`, so the schema is current before the app serves traffic. In Azure the image's plain `CMD` runs and migrations are separate `migrate-<svc>` jobs.

- **Settings:** `env_file: .env` loads every setting; per-service `environment` adds `SERVICE_NAME`, `PYTHONPATH` and the OTLP endpoint.
- The gateway and web have **no bind mounts**: after changing them, run `docker compose up -d --build gateway` (or `web`).

Start-up order and health

Container	Waits for (<code>depends_on</code>)
identity, search, notification	postgres healthy , redis healthy , servicebus started
listing	the above + azurite started
ai	the above + azurite and cosmos started
engagement	the above + cosmos started
gateway	redis healthy, identity started
servicebus	mssql started
web	gateway started

Only `postgres` (`pg_isready` , every 5 s, 20 tries) and `redis` (`redis-cli ping`) have Docker health checks. "Started" doesn't mean ready, so the code copes with slow emulators itself: Service Bus consumers and outbox relays retry every 5 s (the emulator needs SQL Server first), engagement retries its Cosmos set-up 30 times at 5-second intervals, and each service reports real readiness at `/health/ready` , which the gateway polls every 10 s. Expect `/health/deep` to say `degraded` for the first minute.

Where data lives (and what survives)

Data	Stored in	<code>docker compose stop / restart</code>	<code>docker compose down</code>	<code>docker compose down -v</code>
Postgres (all 4 databases)	named volume <code>pg-data</code>	kept	kept	deleted
Photos and PDFs	named volume <code>azurite-data</code>	kept	kept	deleted

Data	Stored in	<code>docker compose stop / restart</code>	<code>docker compose down</code>	<code>docker compose down -v</code>
Redis (cache, revocation, flags, rate limits)	container filesystem (append-only file, no volume)	kept	lost	lost
Cosmos (enquiries, favourites, saved searches, AI log, product events)	container filesystem (no volume)	kept	lost	lost
Service Bus messages	mssql container (no volume)	kept	lost	lost

So after a plain `docker compose down`, listings remain but enquiries and AI logs are gone. `infra/local/postgres/init.sql` runs only when `pg-data` is empty: it creates the four databases and enables the `vector` extension in `search` and `ai`. `infra/local/servicebus/Config.json` is mounted into the emulator and read at start (restart `servicebus` after editing it).

Everyday commands

Goal	Command
Start or update everything	<code>docker compose up -d --build</code>
See what's running and healthy	<code>docker compose ps</code>
Follow one service's logs	<code>docker compose logs -f listing</code>
Shell inside a container	<code>docker compose exec listing sh</code>
Run a one-off command in a service	<code>docker compose exec listing alembic current</code>
Rebuild one image (gateway, web, or after a requirements change)	<code>docker compose up -d --build <service></code>
Restart after editing <code>Config.json</code>	<code>docker compose restart servicebus</code>
CPU and memory per container	<code>docker stats</code>

Goal	Command
Stop (keep everything)	<code>docker compose stop</code>
Remove containers, keep Postgres and blobs	<code>docker compose down</code>
Wipe everything	<code>docker compose down -v</code>

From compose to Azure

The same images run in Azure Container Apps without the dev overrides: no bind mounts, no `--reload`, the plain `CMD`, settings from Key Vault references instead of `.env`, migrations as jobs, and managed services instead of the eight emulator containers (`infra/azure/`, section 18). The service names stay the same (`http://identity`, `http://listing`, ... on Container Apps' internal network), which is why the code needs no changes between the two.

16. Debugging guide

Start every investigation from the **request id**: it is in the `X-Request-Id` response header, in every error body (`error.request_id`), on every log line of every service the request touched, and on the events it caused (`correlation_id`). With it you can follow one click across the gateway, the services and Service Bus.

16.1 Follow one request

1. In the browser, open DevTools → Network, click the failing `/api/v1/...` call and copy the `X-Request-Id` response header (or `request_id` from the JSON error).
2. Find every log line for it: `docker compose logs --no-color --since 30m | grep <request id>` (PowerShell: `docker compose logs --no-color --since 30m | Select-String <request id>`).
3. Open the Aspire Dashboard at `http://localhost:18888` → Traces. The gateway span, the service span and its outgoing HTTP calls (to ai, listing, ...) show as one waterfall with timings. Structured Logs there are filterable by `request_id` .
4. Each Python service logs one `request` line per call with `method`, `path`, `status`, `duration_ms` . Errors log `unhandled_error` with a stack trace; domain errors (404, 422, ...) are expected and only appear in the `request` line.

Useful log events to search for

Event	Service	Means
<code>consumer_started</code> / <code>consumer_connection_failed_retrying</code>	any consumer	Subscribed to a topic / Service Bus not ready yet
<code>event_handled</code> / <code>event_handler_failed</code>	any consumer	Message processed / handler raised (it will be retried, then dead-lettered)
<code>outbox_published</code> / <code>outbox_publish_failed</code>	listing, ai, identity	Events left the outbox / Service Bus down, will retry
<code>embedding_failed_indexing_without_vector</code>	search	ai was unreachable; listing indexed for filters and full-text only
<code>nl_parse_failed_using_fallback</code>	search	Gemini parse failed or timed out; results are "broad matches"
<code>ai_request</code>	ai	One LLM call: feature, model, status, latency, tokens
<code>fair_housing_check_unavailable</code>	listing	Publish allowed without the wording check
<code>document_ingested</code>	ai	PDF chunked and embedded
<code>enquiry_email_sent</code>	notification	Agent emailed
<code>Retrying {Upstream}</code> / <code>Circuit opened for {Upstream}</code>	gateway	Upstream errors; while open, calls fail fast with 503

16.2 Look inside the data

- **Postgres:** `docker compose exec postgres psql -U estate -d listing` (or `identity`, `search`, `ai`). Handy queries: `select status, count(*) from listings group by 1;` — `select count(*), max(attempts), max(last_error) from outbox where published_at is null;` (stuck events) — `select id, title from search_listings where title ilike '%kharadi%';` (is it indexed?) — `select document_id, count(*) from document_chunks group by 1;` (in `ai`).
- **Redis:** `docker compose exec redis redis-cli` then `KEYS rl:*` (rate-limit windows), `GET jwt:revoked_before:<user id>`, `HGETALL feature_flags`, `KEYS idem:*`, `MONITOR` (live command stream; stop with Ctrl+C).
- **Cosmos:** Data Explorer at `http://localhost:1234` → database `estateai` → `enquiries`, `ai-requests` (each AI call with redacted prompt/response, tokens, cost, feedback), `product-events`.
- **Blob:** Azure Storage Explorer connected to Azurite (`UseDevelopmentStorage=true`), container `listing-media`.
- **Email:** Mailpit at `http://localhost:8025` shows every sign-in and enquiry email.
- **Service Bus:** the emulator has no explorer. Check the producer's outbox (above), the consumer's `event_handled` / `event_handler_failed` logs, and `docker compose logs servicebus` for topic errors. A topic missing after editing `Config.json` means the emulator wasn't restarted.

16.3 Set breakpoints in a Python service

Option A — attach to the container (no setup change for other services). Create a `docker-compose.debug.yml` (don't commit it) for the service you want, e.g. `listing`:

```
services:
  listing:
    ports: ["5678:5678"]
    command: sh -c "pip install debugpy && alembic upgrade head && python -m
debugpy --listen 0.0.0.0:5678 -m uvicorn app.main:app --host 0.0.0.0 --port
8000"
```

Start with `docker compose -f docker-compose.yml -f docker-compose.debug.yml up -d listing`. In VS Code add an "attach" configuration to `.vscode/launch.json`:

```
{ "name": "Attach listing", "type": "debugpy", "request": "attach",
  "connect": { "host": "localhost", "port": 5678 },
  "pathMappings": [ { "localRoot": "${workspaceFolder}/services/listing/app",
    "remoteRoot": "/app/app" },
    { "localRoot": "${workspaceFolder}/libs/common",
    "remoteRoot": "/libs/common" } ] }
```

Set a breakpoint (for example in `publish_listing` in `services/listing/app/main.py`), press F5, and click Publish in the app. Breakpoints also work in background code: consumers (`on_listing_event` in `search`, `on_identity_event` in `listing`), the outbox relay and the ingestion pipeline. This setup has no auto-reload; restart the container after code changes.

Option B — run the service on your machine. Stop its container (`docker compose stop search`), then from `services/search` run it under your debugger with these overrides (PowerShell `$env:NAME="..."`): `PYTHONPATH=../../libs/common;. , DATABASE_URL=postgresql+asyncpg://estate:estate@localhost:5432/search , REDIS_URL=redis://localhost:6379/0 , AI_URL=http://localhost:8004 , SERVICEBUS_CONNECTION with Endpoint=sb://localhost;... , and uvicorn app.main:app --port 8003 . Other services and the gateway still call search:8000 inside Docker, so test it directly on port 8003 (/docs gives a Swagger UI) or point the gateway at it with ReverseProxy__Clusters__search__Destinations__d1__Address=http://host.docker.internal:8003/ .`

Option C — debug through a unit test. Tests need no Docker: `cd services/listing; pytest -q -k publish --pdb` stops at the failure, or put `breakpoint()` in the code. VS Code's Testing panel can debug a single test once the interpreter has the requirements installed.

16.4 Debug the gateway (C#)

- Routes, policies and clusters are all in `services/gateway/src/Gateway/appsettings.json` ; most "404 Unknown route" or "403" surprises are a missing or mis-matched route there. Remember to rebuild the gateway container after editing it.
- Logs are JSON on stdout: `docker compose logs -f gateway` .
- `GET http://localhost:8080/health/deep` shows each cluster as Healthy/Unhealthy as seen by the gateway.

- To step through it: `docker compose stop gateway`, then run `dotnet run --project services/gateway/src/Gateway` from Visual Studio, Rider or VS Code (C# Dev Kit) with `Redis__ConnectionString=localhost:6379`, `Jwt__MetadataAddress=http://localhost:8001/.well-known/openid-configuration`, `Jwt__RequireHttpsMetadata=false` and each `ReverseProxy__Clusters__<svc>__Destinations__d1__Address=http://localhost:800N/`. Good breakpoints: `EdgeTokenMiddleware`, the rate-limit partition lambda in `RateLimiting.cs`, `ResilientForwarderHttpClientFactory`.
- The 19 gateway tests (`cd services/gateway; dotnet test`) run the real pipeline against an in-process upstream and are the quickest way to debug a policy.

16.5 Debug the frontend

- Run `cd web; npm run dev` and use `http://localhost:5173`: hot reload and source maps, so breakpoints in DevTools → Sources land on your `.tsx` lines. `debugger`; statements work too.
- Network tab: every call is `/api/v1/...`; failed ones show the error envelope. A 401 followed by `/auth/refresh` and a retry is normal.
- Q&A streaming: the `/qa` request's response tab shows the SSE `event: / data:` lines as they arrive.
- State lives in the URL on the search page, so copying the URL reproduces exactly what someone saw.
- Playwright: `npx playwright test --project=chromium --debug` steps through a test; `--ui` gives a time-travel UI; failed runs leave `test-results/<test>/trace.zip` for `npx playwright show-trace <file>`.

16.6 Debug the AI features

- With `AI_FAKE=true` nothing calls Gemini; answers come from `llm/fake.py`, which is deterministic, so a wrong answer there is a bug in your code, not the model.
- Every real or fake call is in Cosmos `ai-requests`: open it to see the redacted prompt, the response, the prompt version, tokens, cost and latency. Admin → AI shows the same aggregated.
- To see a rendered prompt without calling anything, from `services/ai` run `python -c "from app.prompts.loader import render; p = render('qa', facts=[{'id': 'S1', 'label': 'Floor', 'value': '3 of 10'}], documents=[],`

`question='Which floor?'); print(p.system); print(p.user)"` (with `PYTHONPATH` set as above).

- Turn a feature off at runtime on Admin → AI to check its fallback, or stop the ai container (`docker compose stop ai`): search must fall back to broad matches and listing pages must keep working (NFR-2).

16.7 Common problems

Symptom	Likely cause	Fix
<code>failed to connect to the docker API ... dockerDesktopLinuxEngine</code>	Docker Desktop isn't running	Start Docker Desktop, wait for it, <code>docker compose up -d</code>
<code>/health/deep</code> says <code>degraded</code> for a minute after start	Service Bus emulator (SQL Server) or Cosmos still starting	Wait; services retry their connections every 5 s
New listing never appears in search	Outbox stuck (Service Bus down), consumer failing, or it isn't <code>published</code>	Outbox query in 16.2; search logs for <code>event_handler_failed</code> ; check the listing's status
A new route returns <code>404 Unknown route</code>	Gateway route missing, or gateway image not rebuilt	Add the route to <code>appsettings.json</code> ; <code>docker compose up -d --build gateway</code>
New event type never arrives	Topic/subscription missing in <code>Config.json</code> , or emulator not restarted	Edit <code>Config.json</code> ; <code>docker compose restart servicebus</code>
<code>429</code> on sign-in during tests	Gateway <code>auth</code> limit (10/min per IP)	Wait for <code>Retry-After</code> , or raise <code>RateLimits__Auth__PermitLimit</code> locally
<code>401</code> right after a role change	The cut-off ended the old token by design	The SPA refreshes automatically; scripts must sign in again

Symptom	Likely cause	Fix
PDF stuck in "Reading..."	ai consumer down, or embeddings failing	ai logs; <code>docker compose restart ai</code> ; failures show a reason in the editor
Map area blank or "The map couldn't load"	No WebGL in that browser, or tiles blocked	Expected fallback; the list still works
<code>UnicodeEncodeError</code> running <code>smoke_test.py</code> on Windows	Console code page isn't UTF-8	<code>\$env:PYTHONIOENCODING="utf-8"</code> (PowerShell) first
CI "api-reference.md is stale"	API or UI call lines moved	<code>python scripts/gen_api_reference.py</code> and commit
Search weights look wrong	Old <code>RANK_W_*</code> values in your <code>.env</code> override the defaults	Copy the block from <code>.env.example</code>

16.8 Worked example: "I published a listing but can't find it"

1. Listing status: `select status, published_at from listings where id='<id>'; in listing`. Not published → the publish call failed; its 422 details say why.
2. Did the event leave? `select event_type, published_at, last_error from outbox order by created_at desc limit 5; — published_at null with an error = Service Bus problem.`
3. Did search receive it? `docker compose logs search | Select-String <listing id>` should show `event_handled`; `event_handler_failed` shows the stack trace.
4. Is it in the read model? `select id, city, locality, lat, lng from search_listings where id='<id>'; in search`.
5. Still missing from results? Your filters, `bbox` or budget exclude it, or a cached page is being served (classic results are cached up to 60 s, 30 s at the gateway).

16a. Debugging with a debugger, step by step

You can put breakpoints in every part of the system at the same time: all six Python services (attached inside their containers), the C# gateway (run from source), and the React app (in Chrome), all from one VS Code window. The Python and gateway set-ups below were tested on the running stack on 2026-10-05: a breakpoint in listing's `get_listing` stopped with `listing_id`, `session` and `user` visible, and one in search's event consumer `on_listing_event` stopped when a re-published listing event arrived.

Step 1 — Install the tools (once)

- **VS Code** with the extensions *Python*, *Python Debugger* (debugpy), *C# Dev Kit* (gateway), *Docker* (optional, to see containers), *Playwright Test for VS Code* (optional).
- **Chrome or Edge** for the web app; optionally the *React Developer Tools* browser extension.
- Open the **repository root** (`D:\Real Estate`) as the VS Code folder, so `${workspaceFolder}` points there.
- Alternatives work the same way: PyCharm Professional (a "Python Debug Server" or Docker Compose interpreter), Visual Studio or Rider for the gateway.

Step 2 — Start the Python services under the debugger

The file `docker-compose.debug.yml` in the repository root (added with this guide) overlays the normal compose file. It runs each Python service under `debugpy` without auto-reload and publishes one debug port per service:

```

services:
  identity:
    ports: ["5701:5678"]
    command: sh -c "pip install --quiet debugpy && alembic upgrade head &&
python -m debugpy --listen 0.0.0.0:5678 -m uvicorn app.main:app --host
0.0.0.0 --port 8000"
  listing:
    ports: ["5702:5678"]
    command: sh -c "pip install --quiet debugpy && alembic upgrade head &&
python -m debugpy --listen 0.0.0.0:5678 -m uvicorn app.main:app --host
0.0.0.0 --port 8000"
  search:      # 5703, same command
  ai:          # 5704, same command
  engagement:  # 5705, same command without alembic
  notification:
    ports: ["5706:5678"]
    command: sh -c "pip install --quiet debugpy && python -m debugpy --listen
0.0.0.0:5678 -m app.main"

```

Service	Debug port	Service port
identity	5701	8001
listing	5702	8002
search	5703	8003
ai	5704	8004
engagement	5705	8005
notification	5706	—

Start them: `docker compose -f docker-compose.yml -f docker-compose.debug.yml up -d identity listing search ai engagement notification`. You can also name just the one you want. The debug ports start at 5701 because 5672 is Service Bus. Go back to normal mode with `docker compose up -d identity listing search ai engagement notification`. In debug mode there is no auto-reload: restart a service (`docker compose -f ... -f docker-compose.debug.yml restart listing`) after editing its code.

Step 3 — Add the launch configurations

Create `.vscode/launch.json` (the folder is git-ignored, so this stays on your machine):

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Attach: identity", "type": "debugpy", "request": "attach",
      "connect": { "host": "localhost", "port": 5701 }, "justMyCode": false,
      "pathMappings": [ { "localRoot":
"${workspaceFolder}/services/identity/app", "remoteRoot": "/app/app" },
        { "localRoot": "${workspaceFolder}/libs/common",
"remoteRoot": "/libs/common" } ] },
    {
      "name": "Attach: listing", "type": "debugpy", "request": "attach",
      "connect": { "host": "localhost", "port": 5702 }, "justMyCode": false,
      "pathMappings": [ { "localRoot":
"${workspaceFolder}/services/listing/app", "remoteRoot": "/app/app" },
        { "localRoot": "${workspaceFolder}/libs/common",
"remoteRoot": "/libs/common" } ] },
    {
      "name": "Attach: search", "type": "debugpy", "request": "attach",
      "connect": { "host": "localhost", "port": 5703 }, "justMyCode": false,
      "pathMappings": [ { "localRoot":
"${workspaceFolder}/services/search/app", "remoteRoot": "/app/app" },
        { "localRoot": "${workspaceFolder}/libs/common",
"remoteRoot": "/libs/common" } ] },
    {
      "name": "Attach: ai", "type": "debugpy", "request": "attach",
      "connect": { "host": "localhost", "port": 5704 }, "justMyCode": false,
      "pathMappings": [ { "localRoot": "${workspaceFolder}/services/ai/app",
"remoteRoot": "/app/app" },
        { "localRoot": "${workspaceFolder}/libs/common",
"remoteRoot": "/libs/common" } ] },
    {
      "name": "Attach: engagement", "type": "debugpy", "request": "attach",
      "connect": { "host": "localhost", "port": 5705 }, "justMyCode": false,
      "pathMappings": [ { "localRoot":
"${workspaceFolder}/services/engagement/app", "remoteRoot": "/app/app" },
        { "localRoot": "${workspaceFolder}/libs/common",
"remoteRoot": "/libs/common" } ] },
    {
      "name": "Attach: notification", "type": "debugpy", "request": "attach",
      "connect": { "host": "localhost", "port": 5706 }, "justMyCode": false,
      "pathMappings": [ { "localRoot":
"${workspaceFolder}/services/notification/app", "remoteRoot": "/app/app" },
        { "localRoot": "${workspaceFolder}/libs/common",
"remoteRoot": "/libs/common" } ] },
  ]
}
```

```

    { "name": "Gateway (run from source)", "type": "coreclr", "request":
"launch", "preLaunchTask": "build gateway",
      "program":
"${workspaceFolder}/services/gateway/src/Gateway/bin/Debug/net10.0/Gateway.dll",
      "cwd": "${workspaceFolder}/services/gateway/src/Gateway",
      "env": {
        "ASPNETCORE_ENVIRONMENT": "Development", "ASPNETCORE_URLS":
"http://localhost:8080",
        "Redis__ConnectionString": "localhost:6379",
        "Jwt__MetadataAddress": "http://localhost:8001/.well-known/openid-
configuration", "Jwt__RequireHttpsMetadata": "false",
        "ReverseProxy__Clusters__identity__Destinations__d1__Address":
"http://localhost:8001/",
        "ReverseProxy__Clusters__listing__Destinations__d1__Address":
"http://localhost:8002/",
        "ReverseProxy__Clusters__search__Destinations__d1__Address":
"http://localhost:8003/",
        "ReverseProxy__Clusters__ai__Destinations__d1__Address":
"http://localhost:8004/",
        "ReverseProxy__Clusters__engagement__Destinations__d1__Address":
"http://localhost:8005/" } },
    { "name": "Web (Chrome, Vite :5173)", "type": "chrome", "request":
"launch", "url": "http://localhost:5173", "webRoot": "${workspaceFolder}/web"
},
    { "name": "Python: debug this test file", "type": "debugpy", "request":
"launch", "module": "pytest",
      "args": ["-q", "${file}"], "cwd": "${fileDirname}/..", "justMyCode":
false }
  ],
  "compounds": [
    { "name": "Attach: all Python services",
      "configurations": ["Attach: identity", "Attach: listing", "Attach:
search", "Attach: ai", "Attach: engagement", "Attach: notification"] }
  ]
}

```

And `.vscode/tasks.json` for the gateway build:

```
{ "version": "2.0.0", "tasks": [ { "label": "build gateway", "type":
"process", "command": "dotnet",
  "args": ["build",
    "${workspaceFolder}/services/gateway/src/Gateway/Gateway.csproj"],
  "problemMatcher": "$msCompile" } ] }
```

Why pathMappings matter. The container runs `/app/app/main.py`; VS Code knows `services/listing/app/main.py`. Without the mapping a breakpoint shows as unverified ("Breakpoint in file that does not exist") and never stops. The second mapping lets you set breakpoints in the shared library (`libs/common/estate_common`) too. `justMyCode: false` lets you step into FastAPI, SQLAlchemy and the Azure SDKs.

Step 4 — Debug the gateway (C#)

1. `docker compose stop gateway` so port 8080 is free.
2. Run **Gateway (run from source)**. It builds, then starts the gateway on `localhost:8080`, pointing at the services' published ports. Tested: `/health/deep` reports every cluster ok and searches return 200 through it.
3. Use the app through the **Vite dev server** on `http://localhost:5173` (`cd web; npm run dev`). The nginx container on `:3000` forwards to the gateway *container*, which is stopped.
4. Good breakpoints: `EdgeTokenMiddleware` (`Infrastructure/EdgeAuthentication.cs`, token and revocation checks); the policy lambda in `Infrastructure/RateLimiting.cs` (which key and limit apply); `ResilientForwarderHttpClientFactory.Build` in `Infrastructure/Resilience.cs` (retries and circuit state); `ErrorEnvelope.WriteAsync` (every gateway-made error).
5. When done: stop debugging, then `docker compose start gateway`.

Step 5 — Debug the web app

1. `cd web; npm run dev` (Vite, port 5173, source maps on).
2. Run **Web (Chrome, Vite :5173)**. VS Code opens Chrome; breakpoints in `.tsx` files bind through Vite's source maps. Alternatively use Chrome DevTools → Sources directly, or a debugger; statement.
3. Useful places: `send()` and `api()` in `web/src/lib/api.ts` (every request, its headers and the 401 refresh retry); `postSSE` in `lib/sse.ts` (each streamed Q&A event); the `queryFn` in `pages/Search.tsx` (which endpoint and parameters a search uses); `save / publish` in `pages/ListingEditor.tsx` (field errors).

4. React Developer Tools shows component props and state; the TanStack Query cache is visible through the components that use it.

Step 6 — Debug tests

- **Python:** open a test file (e.g. `services/listing/tests/test_rules.py`), set a breakpoint, run **Python: debug this test file**. Or use VS Code's Testing panel ("Debug Test" on one test), or `pytest -q -k name --pdb` in a terminal. No Docker is needed.
- **Gateway:** in VS Code's Testing panel (C# Dev Kit), "Debug Test" on a test in `tests/Gateway.Tests/GatewayTests.cs`; it runs the real pipeline in-process, so breakpoints in `Program.cs` and `Infrastructure/*` are hit.
- **Browser tests:** `cd web; npx playwright test e2e/journeys.spec.ts --project=chromium --debug` opens the Playwright Inspector and steps line by line; `--ui` gives a time-travel view; the Playwright VS Code extension can set breakpoints in specs.

Step 7 — Follow one action across services

Attach **all Python services** (the compound), then set these breakpoints and publish a listing from the editor. Execution stops in this order:

#	Service	File → function	What you can inspect
1	listing	<code>app/main.py</code> → <code>publish_listing</code>	The listing row, the signed-in <code>user</code>
2	listing	<code>app/rules.py</code> → <code>publish_problems</code>	Which fields are missing
3	ai	<code>app/main.py</code> → <code>internal_listing_text_check</code>	Title and description being checked for fair housing
4	listing	<code>libs/common/estate_common/outbox.py</code> → <code>add_event</code>	The event staged in the same transaction
5	listing	<code>estate_common/outbox.py</code> → <code>OutboxRelay.publish_pending</code>	Rows being sent to Service Bus
6	search	<code>app/main.py</code> → <code>on_listing_event</code>	The event as received (<code>event.type</code> , <code>event.data</code>)

#	Service	File → function	What you can inspect
7	ai	app/main.py → internal_embeddings	The text being embedded for search
8	search	back in on_listing_event → the upsert	The row written to search_listings

For a **Q&A answer**: ai listing_qa → fetch_facts → listing internal_facts → ai features/qa.py answer (question after redaction, retrieved chunks, the rendered prompt) → llm/fake.py stream (or gemini_client.py). For an **enquiry**: engagement create_enquiry → publish_enquiry_event → notification on_enquiry_created. For **sign-in**: identity request_otp → verify_otp → _session_for → tokens.issue_access_token.

Things that behave differently while paused

- **One breakpoint freezes the whole service.** Each Python service is one async event loop, so while it is paused it also stops answering /health/ready. The gateway marks it unhealthy after a few failed probes (every 10 s) and returns 503 for it. Long sessions are easier calling the service's own port (8001-8005, Swagger at /docs) instead of going through the gateway.
- **Gateway timeouts:** a request through the gateway is cut off after 30 s (90 s for AI routes) with 504, even though your breakpoint is still holding it.
- **Message locks:** a consumer paused longer than its subscription's lock (1 minute; 5 minutes for ai on listing-events) loses the message, and Service Bus delivers it again. You'll see the event twice; the handlers are idempotent, so the data stays correct.
- **Tokens:** access tokens live 15 minutes; after a long pause the SPA silently refreshes.
- **Conditional breakpoints** narrow a busy service to one request: right-click a breakpoint → Edit Condition, e.g. `str(listing_id) == '125eddb1-0dc4-4d6e-bce3-40abda637a79'`. **Logpoints** (`listing {listing_id} by {user}`) print without stopping, which keeps health checks passing.
- **Async stepping:** "Step Over" on an `await` line runs the awaited call and stops on the next line; "Step Into" enters the coroutine.
- **Migrations:** to debug one, run it in a one-off container under debugpy: `docker compose run --rm -p 5707:5678 listing python -m debugpy --listen 0.0.0.0:5678 --wait-for-client -m alembic upgrade head`, then attach a copy of "Attach: listing" on

port 5707, adding the mapping `services/listing/migrations` → `/app/migrations`.

17. Testing and CI

Nine layers of checks run on every push and pull request (`.github/workflows/ci.yml`); all of them also run on a laptop.

Layer	Command	What it covers	Count (2026-10-05)
Lint	<code>ruff check services libs evals scripts --select E,F,B --ignore E501,B008</code>	Python errors and likely bugs	clean
Python unit tests	<code>cd services/<svc>; pytest -q (and libs/common)</code>	Rules, guardrails, tokens, sign-in, money, images (incl. HEIC), ranking, POIs, privacy, metrics rollups, flags, cache, idempotency, HTTP resilience, Gemini client with a mocked SDK. No Docker; AI always fake	126
Migrations	CI job	<code>alembic upgrade head → downgrade base → upgrade head</code> for all four databases on real pgvector	4 services

Layer	Command	What it covers	Count (2026-10-05)
Gateway tests	<code>cd services/gateway; dotnet test</code>	The real pipeline against an in-process upstream: internal routes blocked, roles at the edge, revoked tokens, rate limits with <code>Retry-After</code> , output cache per query, retries, 503 envelope, every new route	19
API contract	<code>python scripts/check_api_sync.py + gen_api_reference.py</code>	Every backend endpoint has a UI caller and a gateway route; every UI call has an endpoint; the generated reference is committed and current	IN SYNC, 72 endpoints
Bicep	<code>az bicep build --file infra/azure/main.bicep</code>	Azure templates compile	compiles
Docker images	<code>docker compose build</code>	Every image builds	—

Layer	Command	What it covers	Count (2026-10-05)
End-to-end smoke	<code>python scripts/smoke_test.py</code>	31 checks through the gateway on the full stack: health, seed → outbox → search, classic/NL/amenity search, sign-in, refresh rotation and reuse rejection, roles, idempotent create, publish blocked per field, publish → searchable, map-area search, PDF ingestion, Q&A with citation, description draft, enquiry → email in Mailpit, favourites, saved search, logout and revocation, web served	31
Browser matrix + accessibility	<code>cd web; npx playwright test</code>	Journeys (J1 NL search → cited answer → ask the agent, chip edit, filters/sort in the URL, map pins, listing facts), axe WCAG 2.1 A/AA on home, search, listing, login, privacy, keyboard skip link, pages without WebGL; Chromium, Firefox, WebKit, Pixel 7, iPhone 14 (Edge with <code>E2E_EDGE=1</code>)	57 passed, 3 skipped

Load tests (not in CI; docs/perf/phase1.md): `python perf/seed_search.py --count 100000` loads synthetic listings into the search read model, then `docker run --rm -i -e BASE_URL=http://host.docker.internal:8003 grafana/k6 run - < perf/k6/search.js` sends 50 requests/s of uncached filter, map, `near` and page-2 searches. Result: p95 150–205 ms against a 300 ms target, 0 errors (it was 323–389 ms before search stopped loading vectors and descriptions). Clean up with `--clean`.

AI evals (`evals/run_evals.py` , not in CI): scored against fixed cases; with real Gemini they cost money, so the expected cost is stated before any run. The search suite has 12 cases; the describe and Q&A suites are still to be written.

Where to add a test: a business rule → a unit test next to the module (`tests/test_rules.py` style); a new public route → a gateway test plus `check_api_sync` ; a cross-service flow → a smoke-test check; a UI journey or accessibility concern → a Playwright spec in `web/e2e/` .

18. Azure deployment and operations

The whole Azure environment is described in Bicep under `infra/azure/` and compiles in CI; it has not been deployed yet because that needs a subscription. Full instructions are in `docs/azure-deployment.md` §4a.

What each module creates

Module	Resources
<code>modules/data.bicep</code>	PostgreSQL Flexible 16 with pgvector allowed and the 4 databases (14-day backups); Cosmos DB serverless + <code>estateai</code> ; Service Bus Standard with all four topics and their subscriptions (TTL 14 days, duplicate window 10 min); Storage (ZRS, private <code>listing-media</code> , soft delete); Azure Cache for Redis (TLS only)
<code>modules/security.bicep</code>	Container Registry; one user-assigned identity for the apps; Key Vault (RBAC, purge protection) holding every connection string and the Gemini key; role assignments (Key Vault Secrets User, AcrPull, Blob Data Contributor, Service Bus Data Owner)

Module	Resources
<code>modules/apps.bicep</code>	Container Apps environment; identity, listing, search, ai, engagement (internal ingress, at least 1 replica because they run consumers and relays, liveness <code>/health</code> and readiness <code>/health/ready</code> probes); gateway (external, 2-10 replicas); notification (scales 0-3 on the Service Bus backlog); <code>migrate-<service></code> jobs
<code>modules/edge.bicep</code>	Static Web App for the SPA; Front Door Standard with <code>/api/*</code> → gateway and <code>/*</code> → SPA on one domain (so the SameSite=Strict cookie works)
<code>modules/monitoring.bicep</code>	Availability tests for the site and <code>/api/health/deep</code> from 5 regions every 5 minutes, and alerts to an on-call email: site or API down (3 of 5 regions failing), 24-hour availability below 99.5% (the NFR-1 error budget, about 3 h 36 min a month), more than 20 failed requests in 15 minutes, average response over 300 ms

Deploying (two passes on first install, because the apps need images in the registry):

1. `az group create -n rg-estateai-prod -l centralindia; export POSTGRES_ADMIN_PASSWORD, GEMINI_API_KEY, ALERT_EMAIL .`
2. `DEPLOY_APPS=false az deployment group create -g rg-estateai-prod -f infra/azure/main.bicep -p infra/azure/main.prod.bicepparam (data, registry, vault, workspace).`
3. Build and push the images with `az acr build` (one per service, tagged with the git SHA).
4. Deploy again with `IMAGE_TAG=<tag>` to create the apps, Front Door and monitoring; run `az containerapp job start -n migrate-<svc>` for each database.
5. Check with `az deployment group what-if` before every later change.

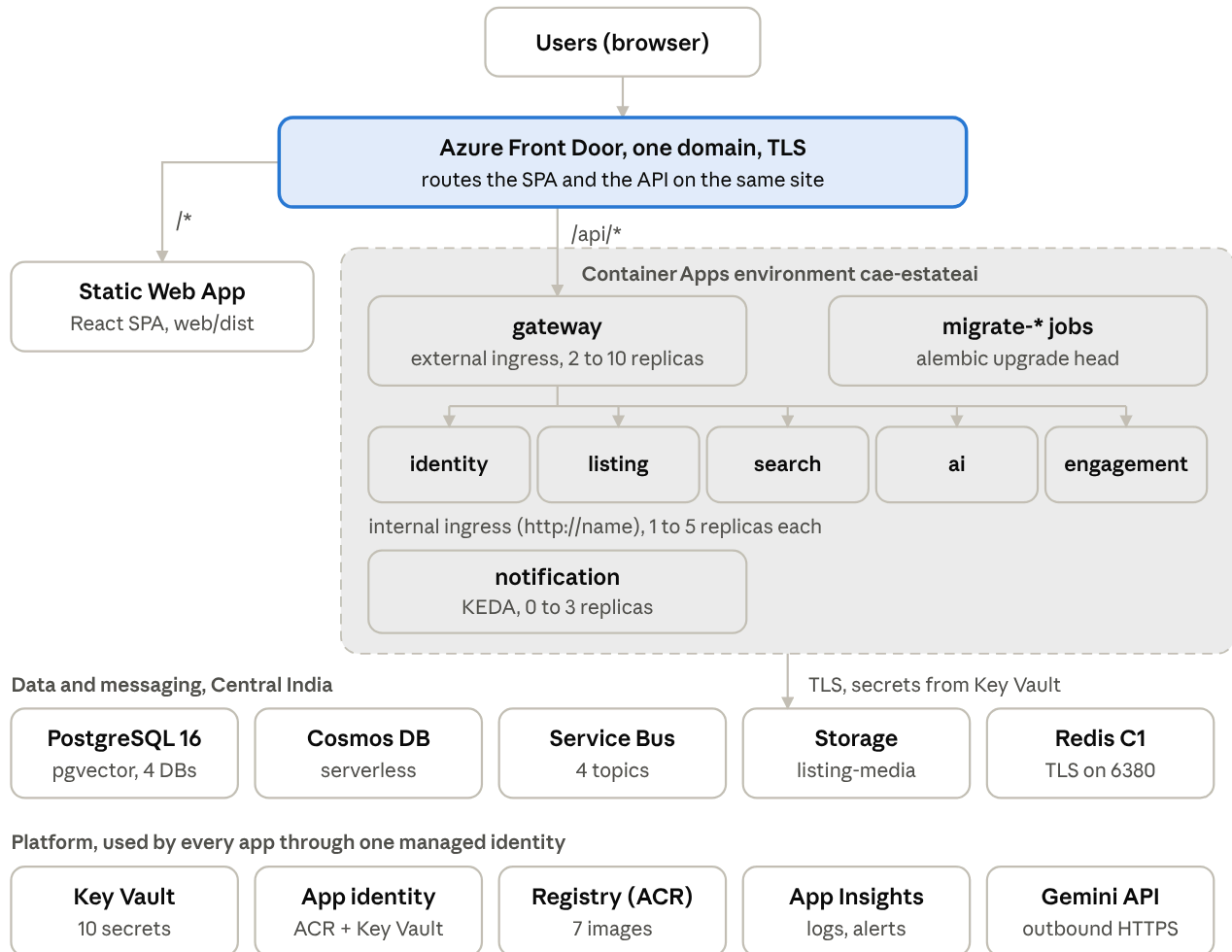
Not automated yet: custom domain and certificate on Front Door, the email domain for Azure Communication Services, VNet integration and private endpoints, signing keys in Key Vault (T4.14), switching Storage/Service Bus/Cosmos to managed identity, and the Container Apps OpenTelemetry agent.

Operating it (`docs/runbook.md`): each alert links to a playbook (site down §6a, error budget §6b, errors §6c, slow §6). Common actions: roll back with `az containerapp revision activate` ; switch an AI feature off on Admin → AI; suspend an abusive agent on Admin → People; check stuck events with the outbox query; rotate signing keys from Admin → People. Rough cost for a small production setup: about \$500–800 a month on Azure plus Gemini usage.

18a. Azure architecture in depth

In Azure the same container images run on Azure Container Apps behind one Front Door domain, the eight emulators are replaced by managed services in Central India, and every secret comes from Key Vault through one managed identity. Everything below is declared in `infra/azure/` (section 18); it has not been deployed yet, so treat sizes and costs as the plan, not measurements.

One Front Door domain fronts the SPA and the gateway



Azure · Front Door, Container Apps, data services, platform

Users reach one Front Door domain: `/*` is served by the Static Web App, `/api/*` by the gateway Container App. The gateway calls the five services over Container Apps' internal network. Every app reads its connection strings from Key Vault through the shared managed identity and talks to the managed data services over TLS.

Resources and their settings

Layer	Resource (name)	Setting in the templates	Replaces locally
Edge	Front Door Standard (<code>afd-estateai</code> , endpoint <code>ep-estateai</code>)	Routes <code>/api/*</code> → gateway (HTTPS only, no caching) and <code>/*</code> → Static Web App (cached, compressed); HTTP redirects to HTTPS; health probes every 60 s	nginx in <code>web</code>
Edge	Static Web App Standard (<code>swa-estateai</code> , East Asia)	Hosts <code>web/dist</code> ; Central India isn't an SWA region	<code>web</code> container
Compute	Container Apps environment (<code>cae-estateai</code>)	Logs to Log Analytics; no VNet yet	Docker network
Compute	Container App <code>gateway</code>	External ingress, target port 8080, 2–10 replicas, readiness probe <code>/health</code>	<code>gateway</code>
Compute	Container Apps <code>identity</code> , <code>listing</code> , <code>search</code> , <code>ai</code> , <code>engagement</code>	Internal ingress (reachable only inside the environment at <code>http://<name></code>), target port 8000, 0.5 vCPU / 1 GiB, 1–5 replicas (scale at 50 concurrent requests each), liveness <code>/health</code> , readiness <code>/health/ready</code> (3 failures)	the five service containers

Layer	Resource (name)	Setting in the templates	Replaces locally
Compute	ContainerApp notification	No ingress, 0.25 vCPU / 0.5 GiB, 0-3 replicas scaled by KEDA on 20 waiting messages in engagement-events/notification	notification
Compute	ContainerApps jobs migrate-identity/listing/search/ai	Manual trigger, alembic upgrade head , 600 s timeout, 1 retry	alembic upgrade head in each compose command
Data	PostgreSQL Flexible 16 (pg-estateai)	Burstable B2s, 64 GB auto-grow, pgvector allowed, 4 databases, 14-day backups, firewall: Azure services only	postgres
Data	Cosmos DB serverless (cosmos-estateai)	Session consistency, TLS 1.2, database estateai (containers created by the services)	cosmos
Messaging	Service Bus Standard (sb-estateai)	4 topics, 7 subscriptions, TTL 14 days, duplicate window 10 min, dead-letter on expiry	servicebus + mssql
Data	Storage account (stestateaimedia)	ZRS, no public blob access, TLS 1.2, private listing-media , 14-day soft delete	azurite
Data	Azure Cache for Redis Standard C1 (redis-estateai)	TLS 1.2 only (port 6380), non-TLS port off	redis

Layer	Resource (name)	Setting in the templates	Replaces locally
Security	Key Vault (<code>kv-estateai</code>)	RBAC, soft delete 30 days, purge protection; 10 secrets	<code>.env</code>
Security	Managed identity <code>id-estateai-apps</code> + Container Registry	Roles: Key Vault Secrets User, AcrPull, Blob Data Contributor, Service Bus Data Owner	—
Monitoring	Log Analytics (<code>law-estateai</code> , 30 days) + Application Insights (<code>appi-estateai</code> , IP masking on)	Availability tests + alerts (section 18)	<code>aspire-dashboard</code>
External	Google Gemini API; Azure Communication Services Email (to add)	Outbound HTTPS; ACS email domain is manual	Gemini, <code>mailpit</code>

A request in Azure, hop by hop

1. DNS for the site (custom domain, or the Front Door endpoint) resolves to Front Door's nearest edge; TLS ends there.
2. Front Door matches the path: `/api/*` goes to origin group `api` (the gateway's public FQDN over HTTPS); anything else goes to `web` (the Static Web App, served from cache when possible).
3. The gateway Container App receives it on 8080. Everything from section 5 happens here, with rate limits, output cache and revocation stored in Azure Cache for Redis.
4. The gateway forwards to `http://listing` (or another service). Container Apps' internal DNS resolves the app name; internal ingress maps port 80 to the target port 8000. This traffic never leaves the environment.
5. The service talks to PostgreSQL over TLS (`ssl=require`), Redis over TLS, Service Bus over AMQP/TLS, Blob and Cosmos over HTTPS, all with connection strings read from Key Vault.
6. Events go through the outbox to Service Bus; subscriber apps receive them on their subscriptions; `notification` scales up from zero when enquiry messages arrive and emails through ACS.

Because the SPA and the API share Front Door's single domain, the `SameSite=Strict` refresh cookie is first-party and there is no CORS.

Identities and secrets

- One user-assigned identity is attached to every app and job. It pulls images from ACR (`AcrPull`) and reads Key Vault (`Key Vault Secrets User`). No registry passwords or vault keys exist anywhere.
- Each app's secrets are **Key Vault references** (`keyVaultUrl` + identity). Container Apps resolves them when a revision starts and exposes them as environment variables through `secretRef` , so the code still just reads `DATABASE_URL` , `REDIS_URL` , etc.
- Key Vault holds: `servicebus-connection` , `storage-connection` , `cosmos-key` , `redis-url` , `redis-gateway-connection` , `gemini-api-key` , `db-identity` , `db-listing` , `db-search` , `db-ai` . The Bicep generates them from the created resources' keys plus the Postgres admin password you pass in.
- **Rotating a secret**: update it in Key Vault, then restart the apps' revisions so they read the new value.
- Prepared but not used yet: the identity also has Blob Data Contributor and Service Bus Data Owner, so the code can switch from connection strings to `DefaultAzureCredential` (azure-deployment.md §3 #8) without infrastructure changes. JWT signing keys still live in Postgres; moving them to Key Vault is task T4.14.

Networking: today and the target

Aspect	In the templates now	Production target
Container Apps environment	Public, no VNet	VNet-integrated (infrastructure subnet)
Gateway ingress	Public FQDN	Accept only Front Door: service tag + <code>X-Azure-FDID</code> header check
Postgres	Public endpoint, firewall "Azure services" only	Private endpoint or VNet integration, public access off
Cosmos, Service Bus, Storage, Redis, Key Vault	Public endpoints with keys/RBAC	Private endpoints, public access off

Aspect	In the templates now	Production target
WAF	None (Front Door Standard)	Front Door Premium with OWASP managed rules + a rate-limit rule
Outbound	Open	Allow only <code>generativelanguage.googleapis.com</code> and Azure endpoints

The templates start public so a first environment comes up quickly; `azure-deployment.md §6` is the checklist for closing it.

Scaling and resilience

- **Gateway:** at least 2 replicas, so one can restart without downtime; up to 10.
- **Services:** at least 1 replica, because each runs background work (Service Bus consumers, the outbox relay, purge and retention loops) that stops if it scales to zero. They add replicas at 50 concurrent requests per replica, up to 5. Several replicas are safe: the outbox relay uses `SKIP LOCKED`, consumers compete on one subscription, caches invalidate across replicas through Redis pub/sub.
- **Notification:** 0–3, driven by queue depth (KEDA Service Bus scaler).
- **Health:** Container Apps restarts a replica whose `/health` fails and takes it out of rotation while `/health/ready` fails; the gateway's own health checks and circuit breakers act on top.
- **Single region** (Central India). Zone redundancy, a second region and RTO/RPO targets are not defined yet.

Deploying a release

1. CI passes on the commit (section 17).
2. Build the seven images in ACR, tagged with the commit SHA (`az acr build`).
3. `az deployment group create ... -p main.prod.bicepparam` with `IMAGE_TAG=<sha>`; Container Apps creates a new **revision** of each changed app.
4. Start the `migrate-*` jobs and wait for success. Migrations must be backward compatible, because the old and new revisions run side by side during the switch.
5. Deploy the SPA to the Static Web App (`web/dist`).
6. Check `https://<site>/api/health/deep`, the availability tests and the smoke test against the site.

7. **Roll back** with `az containerapp revision activate --revision <previous>` (instant, no rebuild).

Configuration differences from local

Setting	Local	Azure
APP_ENV	local	prod: no /docs , no dev endpoints or dev users, no on-screen sign-in code, Secure cookies
AI_FAKE / embeddings	true / fake	false / gemini, key from Key Vault
Redis	redis://redis:6379/0	rediss://...:6380/0 (TLS)
Postgres	...@postgres:5432/<db>	...@pg-estateai.postgres.database.azure.com:5432/<db>?ssl=require
Email	Mailpit SMTP	ACS Email (code change still needed for SMTP auth/TLS)
Telemetry	Aspire via OTEL_EXPORTER_OTLP_ENDPOINT	Container Apps OpenTelemetry agent → App Insights (to configure)
Gateway JWT metadata	http://identity:8000/...	http://identity/... (internal, RequireHttpsMetadata=false)

Backups and recovery

Data	Protection in the templates
Postgres	Automatic backups, 14-day point-in-time restore (not geo-redundant)
Blob (photos, PDFs)	ZRS (3 zones) + 14-day soft delete
Cosmos	Default periodic backup (continuous backup not enabled)
Key Vault	30-day soft delete + purge protection
Search read model, document chunks	Rebuildable: re-publish listings (/internal/dev/republish equivalent) and re-upload PDFs

Data	Protection in the templates
Redis	Disposable cache and short-lived security state

A quarterly backup-restore drill is in the runbook (task T4.10, still to do).

Debugging the app in Azure

There is no debugger attached in production; you debug from logs, traces and safe changes.

- **Follow a request id** in Log Analytics (portal → `law-estateai` → Logs):

```
ContainerAppConsoleLogs_CL
| where TimeGenerated > ago(1h)
| where Log_s has "<request id>"
| project TimeGenerated, ContainerAppName_s, RevisionName_s, Log_s
| order by TimeGenerated asc
```

- **Live logs** of one app: `az containerapp logs show -g rg-estateai-prod -n listing --follow --tail 100` (add `--type system` for start-up and probe failures).
- **Shell into a running replica**: `az containerapp exec -g rg-estateai-prod -n listing --command sh`. From there you can run `alembic current` or test connectivity, e.g. `python -c "import httpx; print(httpx.get('http://ai/health/ready').text)"`.
- **Revisions and replicas**: `az containerapp revision list -n listing -g rg-estateai-prod -o table` shows which revision is active and healthy; `az containerapp replica list` shows restarts.
- **Traces**: Application Insights → Transaction search (by operation id or request id) and the Application map (once the OpenTelemetry agent is configured).
- **Events**: Service Bus → topic → subscription → Service Bus Explorer to peek messages and the dead-letter queue; check stuck outbox rows with `az postgres flexible-server connect` and the query from 16.2.
- **Data**: Cosmos Data Explorer in the portal; Storage browser for blobs.
- **More detail temporarily**: `az containerapp update -n listing --set-env-vars LOG_LEVEL=DEBUG` creates a new revision with debug logs; set it back afterwards.
- **Contain a bad feature without a deploy**: switch it off on Admin → AI (section 11), or roll back the revision.

- **Reproduce locally with the exact build:** pull the same image tag from ACR (`az acr login -n <registry>; docker pull <registry>.azurecr.io/listing:<sha>`) and run it in the local stack in place of the built one, then use section 16a's debugger.

19. Glossary and further reading

Term	Meaning
BHK	Bedrooms, Hall, Kitchen: "2 BHK" = two bedrooms
Lakh / crore	1 lakh = ₹1,00,000; 1 crore = ₹1,00,00,000 (shown as ₹80 L , ₹1.25 Cr)
Paise	1/100 of a rupee; all money is stored as integer paise (*_minor)
Carpet / built-up area	Usable floor area / area including walls and balconies; carpet can't exceed built-up
RERA	Real Estate (Regulation and Development) Act; projects sold before completion must show a registration number
DPDP Act 2023	India's data protection law: consent, purpose limitation, right to erasure, breach notification
Fair housing	No listing or AI text may prefer or exclude people by religion, caste, community, gender, marital status or diet
Read model / projection	Search's own copy of published listings, built from events
Outbox	Table where an event is saved in the same transaction as the change, then published by a relay
Idempotent	Doing it twice has the same effect as once
RAG	Retrieval-augmented generation: find relevant document passages, then let the model answer only from them

Term	Meaning
Embedding	A list of 768 numbers that places a text so similar meanings sit close together
SSE	Server-Sent Events: the server streams <code>event: / data:</code> lines over one HTTP response
JWKS	The identity service's public keys, published so others can verify its tokens
AI_FAKE	Offline, deterministic stand-in for Gemini used locally and in all tests
POI	Point of interest (metro station, IT park...) used for <code>near</code> ranking
bbox	Bounding box <code>minLng,minLat,maxLng,maxLat</code> : the visible map area

Where to read next (all in the repository)

Need	Read
Rules for changing code	<code>CLAUDE.md</code>
What the product must do, with IDs (FR-x, NFR-x)	<code>docs/requirements.md</code>
Detailed design, contracts, AI pipelines	<code>docs/design.md</code>
Service catalogue, events, ports	<code>docs/architecture.md</code>
Every endpoint with handler, auth, gateway route and UI caller	<code>docs/api-reference.md</code> (generated)
How to write service code	<code>docs/backend-guide.md</code>
Why each technology was chosen	<code>docs/decisions/0001-0019</code>
Prompts, guardrails, evals	<code>docs/ai/prompts.md</code> , <code>guardrails.md</code> , <code>evals.md</code>
Security rules	<code>docs/security.md</code>
Task status, what's left, parking lot	<code>docs/tasks.md</code>

Need	Read
Operations	<code>docs/runbook.md</code> , <code>docs/azure-deployment.md</code>
Load-test results	<code>docs/perf/phase1.md</code>

Still open (from `docs/tasks.md`): AI eval datasets for describe and Q&A (and growing search to 150 cases); real-Gemini smoke test; Lighthouse page-load check on 4G and a manual screen-reader pass; failure and backup-restore drills once Azure exists; private networking, custom domain, ACS email and Key Vault signing keys in Azure; map pin clustering.